

Introduction to P&R

Synopsys University Program

WEEK 1 : Introduction to P&R

WEEK 2 : Create NDM

CONFIDENTIAL INFORMATION

The information contained in this presentation is the confidential and proprietary information of Synopsys. You are not permitted to disseminate or use any of the information provided to you in this presentation outside of Synopsys without prior written authorization.

IMPORTANT NOTICE

In the event information in this presentation reflects Synopsys' future plans, such plans are as of the date of this presentation and are subject to change. Synopsys is not obligated to update this presentation or develop the products with the features and functionality discussed in this presentation. Additionally, Synopsys' services and products may only be offered and purchased pursuant to an authorized quote and purchase order or a mutually agreed upon written contract with Synopsys.

CONTENTS

1. Introduction to P&R flow

- P&R Flow

2. APPENDIX

- Introduction to (ICC2) IC compiler II
- Directory Structure
- Backend Input files
- Cells & Layer

3. LAB Week 1

- The basic overview of ICC2 Tool

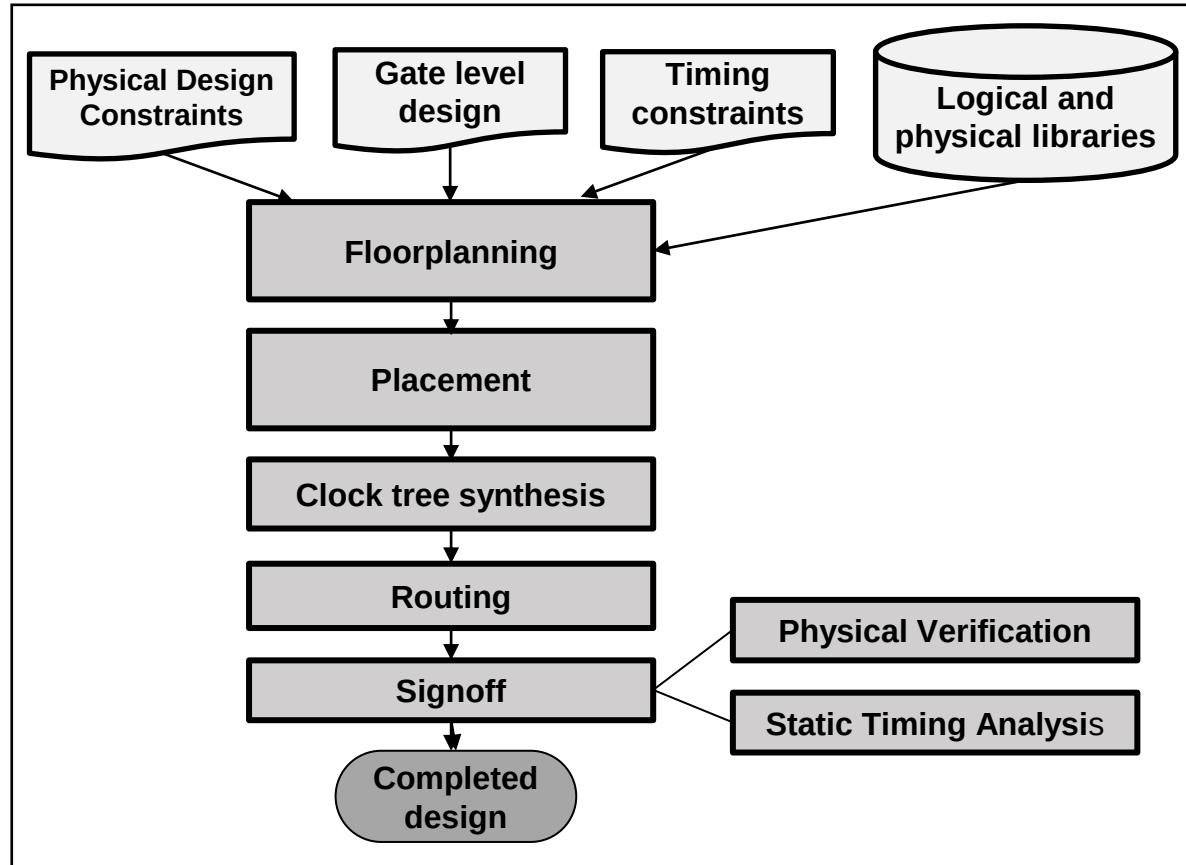
4. LAB Week 2

- Prepare NDM & Create NDM

WEEK 1 : Introduction to P&R flow

P&R Flow : Introduction to P&R

- The backend process is responsible for the physical implementation of a circuit
- P&R transforms the gate circuit into a physical design
- The steps of the backend flow consist of read_design, floorplan, powerplan, placement, CTS, route, in order



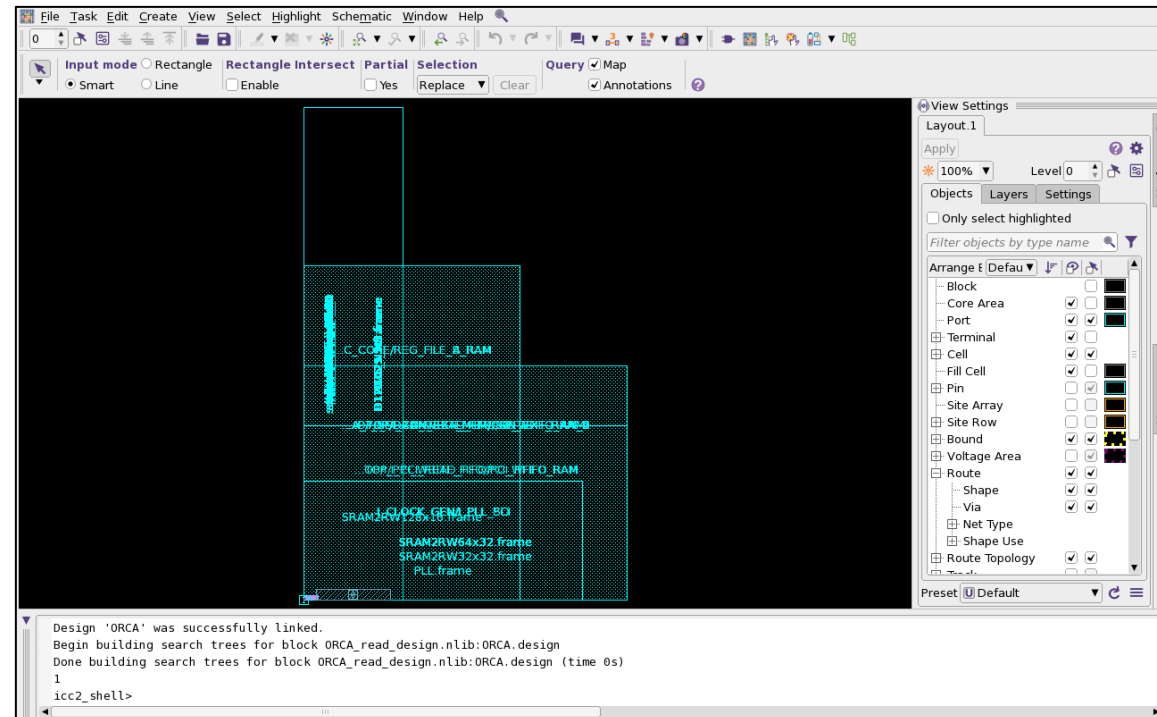
P&R Flow : Design setup & Create NDM

- Why should we use NDM
 - Milkyway library was used in ICC1, but in ICC2, we called it as NDM (New data model)
 - Milkyway is a Synopsys library format that stores all of circuit files from synthesis through place and route all the way to signoff
- What is Design Setup
 - This is the step of setting the necessary things for the backend flow to be performed in ICC2 before starting the flow
 - In this process, after setting design name and .nlib name, designate the path of the directory described above and the paths of constraint files inside each directory. In addition, horizontal and vertical directions of Routing Layers used in ICC2 tool, MIN, MAX Routing Layer, and chip size information are also included
- Goal of Design Setup
 - In all flows, such as the path of input files in the backend flow, about Routing Layers, Design name, etc., input files suitable for each flow are used, or design setup is performed to proceed with the flow with the same set-up applied

P&R Flow : Read design

- What is Read design
 - The read design step refers to the process of loading physical design input files set during design setup before floorplan
- Goal of Read design
 - Load necessary cells, timing information, and input files for each process from floor plan to route in the ICC2 tool and upload them into the tool
- Read design input files

- Netlist
- NDM
- Technology file



Read Physical design input files

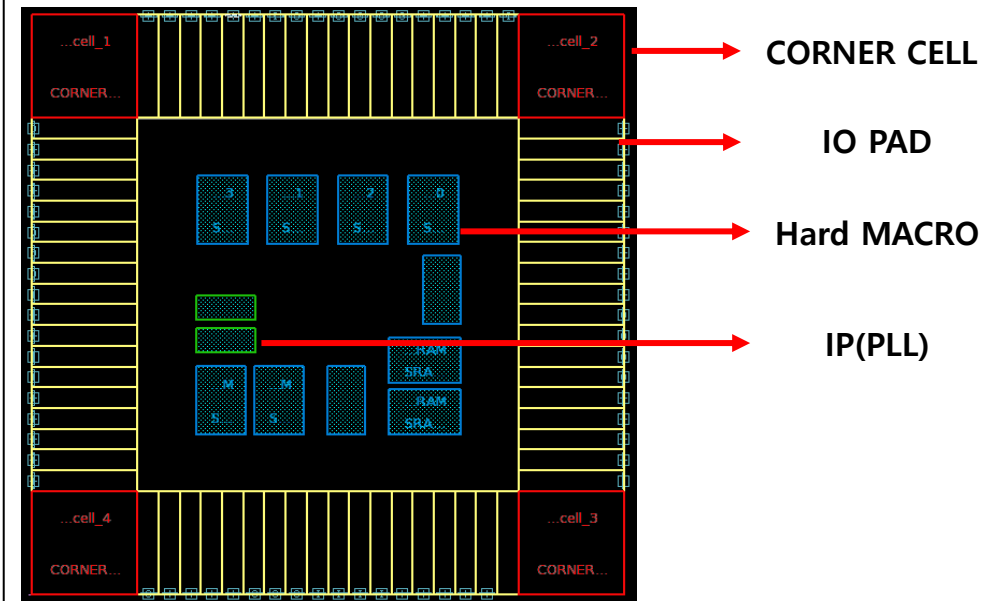
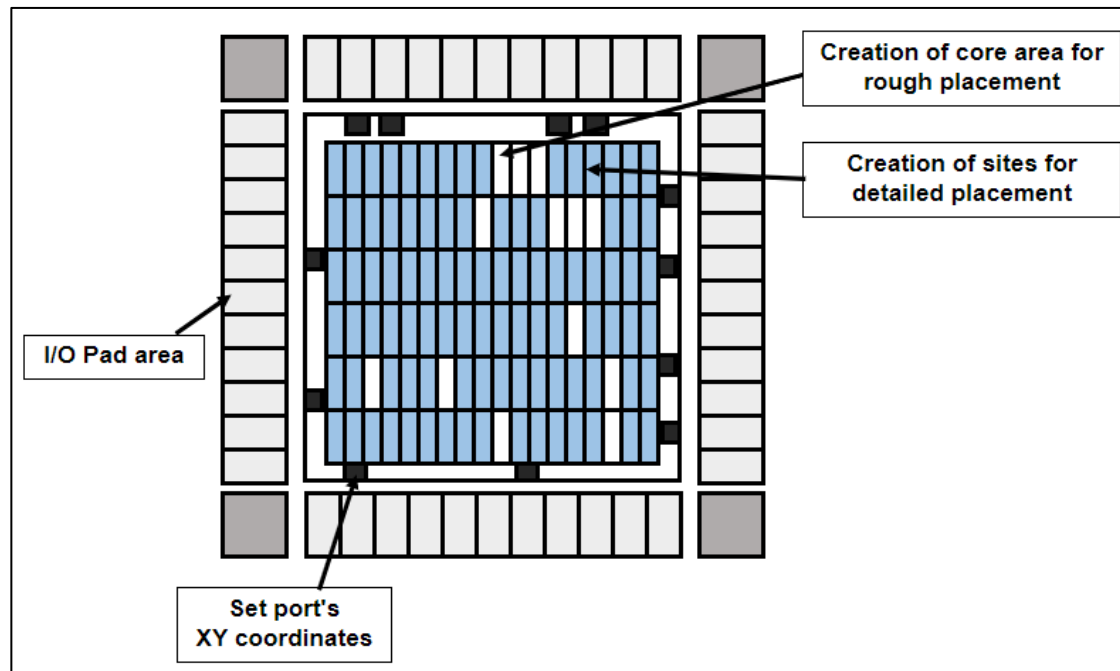
P&R Flow : Floorplan

- **What is Floorplan**

Floorplan is the process of partitioning logical blocks into physical blocks, sizing and placing the blocks, performing a floorplan-level placement of macros, IOs and standard cells, and creating a powerplan

- **Goal of floorplan (Routability, Timing, Wire length, Area for standard cells)**

The goal of floorplan is to increase the efficiency of downstream physical design steps to enable a robust, optimized design an effective floorplan helps to achieve timing closure by placing blocks to minimize critical path length and reduce routing congestion the challenge is to create a floorplan with good area efficiency and a small footprint, while leaving sufficient area for routing



Placement macros, IO, standard cells,

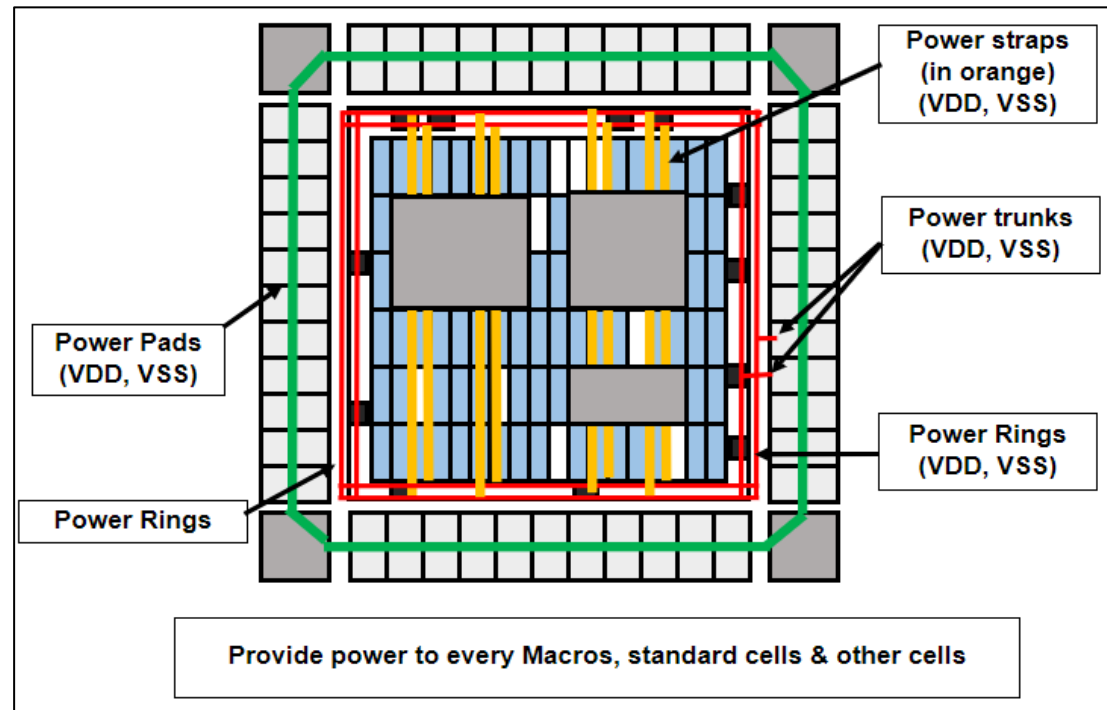
P&R Flow : Powerplan

- **What is powerplan**

- Powerplan is to provide power to every macros, standard cells, and all other cells are present in the design like IO
- Pre-Routing includes creating Power Ring, Stripes/MESH/Grid, and Standard Cell Power Rails and Via insertion
- IO Rings are established through IO Cell abutment and through IO Filler Cells. Power Trunks are constructed between Core Power Ring and Power Pads

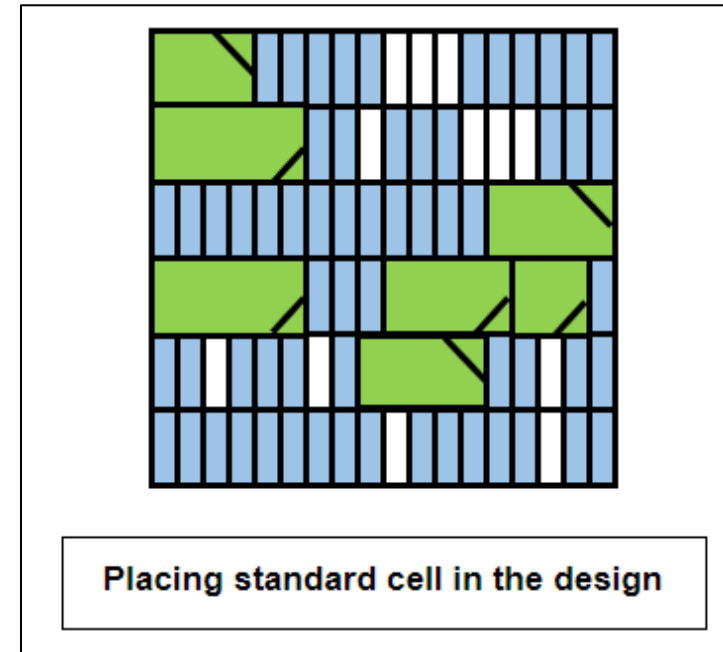
- **Goal of powerplan**

- Powerplan is to connect power to the chip by considering issues like EM and IR DROP (Power Routing also called Pre-Routing)



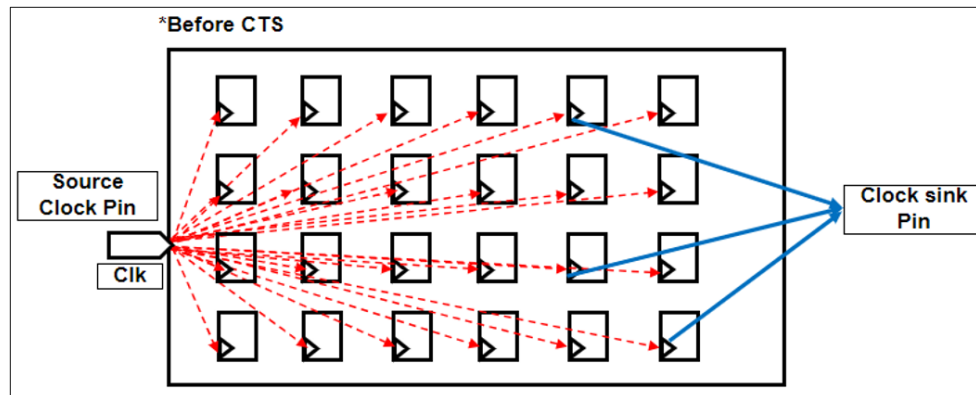
P&R Flow : Placement

- What is placement
 - Placement is the process of placing standard cell in the design and determines the location of each standard cell on the die
(Placement is the process of finding a suitable physical location for each cell in the block)
 - Placement does place the standard cells available in the synthesized netlist and optimizes the design and also determines the routability of design
 - Placement will be driven by different criteria like timing driven, congestion driven and power optimization
 - Automated Standard Cell Placement for placing the Standard Cells in Placement Tracks
- Goal of placement
 - Routable design
 - Minimal timing DRCs
 - Minimal cell density and pin density
 - Reduce the congestion due to cells and pins

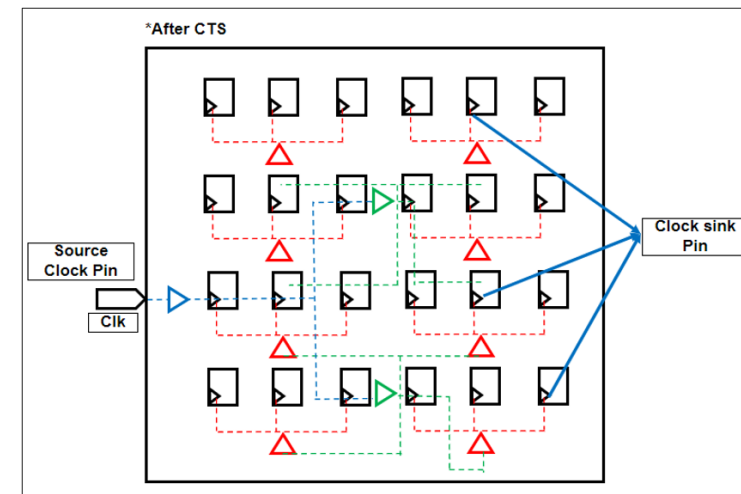


P&R Flow : CTS

- What is CTS
 - The process of distributing the clock and balancing the load is called CTS and delivering the clock to all sequential elements
 - CTS is the process of insertion of buffers or inverters along the clock paths of ASIC design in order to achieve zero/minimum skew or balanced skew.
- Goal of CTS
 - The goal of CTS is to minimize skew and insertion delay
 - Meeting the clock tree targets (Min skew, Min/Max insertion delay(latency))
 - Meeting the clock tree design rule constraints (Max transition delay, Max load capacitance, Max fanout, Max buffer levels)



Distributing the clock and balancing the load



A buffer tree is built to balance the loads and minimize the skew

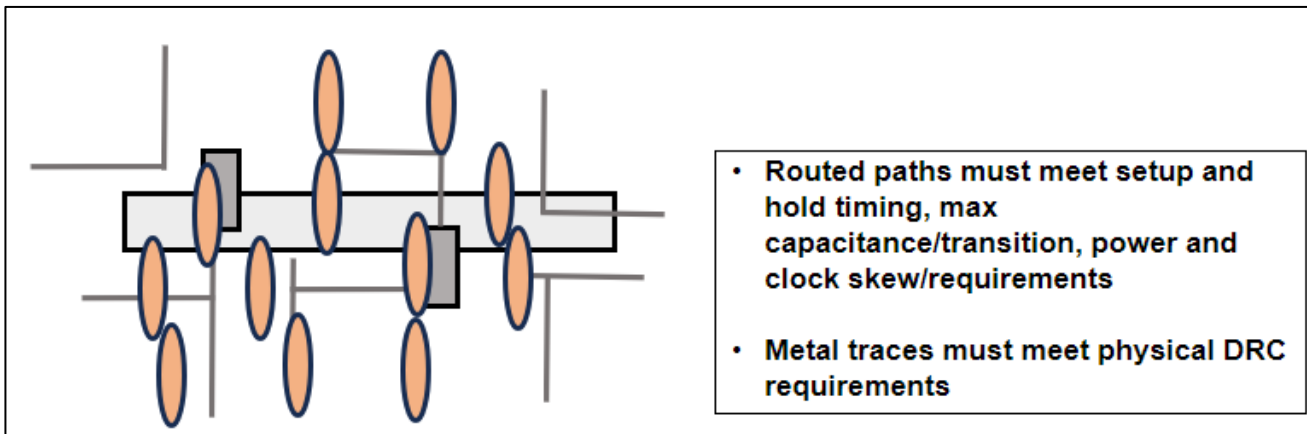
P&R Flow : Route

- What is Route

- Routing in ASIC is the process of making physical connections between signal pins using metal layers
- Following Clock Tree Synthesis (CTS) and optimization, the routing step determines the exact pathways for interconnecting standard cells, macros, and I/O pins
- The layout creates electrical connections using metals and vias that are determined by the logical connections in the netlist

- Goal of Route

- Minimize the total interconnect/wire length and the critical path delay
- Minimize the number of layer changes that the connections have to make (minimizing the number vias)
- Complete the connections without increasing the total area of the block
- Meeting the Timing DRCs and obtaining a good Timing QoR
- Minimizing the congestion hotspots



P&R Flow : PV (Physical Verification)

- What is PV (Physical Verification)
 - PV is a critical step in the chip design process that ensures the correctness and reliability of the physical layout of IC
- DRC
 - DRC stands for Design Rule Check, and it is a physical verification check in ASIC design
 - DRC is the process of checking physical layout data against fabrication-specific rules specified by the foundry to ensure successful fabrication
- LVS
 - LVS stands for Layout Versus Schematic, and it is a physical verification check in ASIC design
 - LVS verifies the connectivity of a Verilog Netlist and Layout Netlist
- IR DROP
 - IR Drop in ASIC refers to the variation in electrical potential between the two ends of a conducting wire when current flows through it.
 - This can affect the design performance of the chip. so every EDA companies have their own IR analysis tool which performs the IR analysis and based on the analysis the techniques for the IR fixes are applied

P&R Flow : PV (Physical Verification)

- Goal of PV (Physical Verification)
 - Physical verification is important in the ASIC design process as it helps identify potential errors, such as incorrect wiring or other layout-related problems, that can cause manufacturing defects, impact the performance of the circuit, or lead to device failure
 - The physical verification process checks for compliance with design rules, such as spacing, metal layers, alignment, via placement, shorts circuits, and open/float pins
 - This procedure in the design ensures that the design meets the manufacturing requirements after clearing all the errors

Introduction to (ICC2) IC compiler II

WEEK 1 : APPENDIX

- Introduction to (ICC2) IC compiler II
- Directory Structure
- Backend Input files
- Cells & Layer

Introduction to (ICC2) IC Compiler II

- What is ICC2
 - ICC2 is the synopsys P&R tool which is a complete netlist-to-GDSII implementation system that includes early design exploration and prototyping, detailed design planning, block implementation, chip assembly and sign-off driven design closure
 - The foundation, architecture and implementation is based on novel, patented technologies and the software has been written using modern object-oriented languages and tools
 - ICC2, which is marketed alongside the original IC Compiler, is meant to give designers of complex ICs, at all process nodes, a ten-fold improvement in design throughput. No single feature can achieve such broad improvements in productivity. Traditional tools produce marginal improvements by applying incremental techniques to an existing architecture
 - Many things that used to do in ICC are no longer valid in ICC2
 - On the other hand, many concepts / used models are carried over to IC Compiler II
 - Similar commands can have subtle or big differences!
 - Always consult the man page
- Benefits of ICC2
 - IC Compiler II benefits from the combination of a new hierarchical infrastructure enabling massive parallelism; a highly compact multi-corner and multi-mode (MCMM) architecture; next-generation design-planning; new global, analytical, and scalable optimization techniques; and global optimization approaches to clock synthesis. IC Compiler II also fits into existing design flows using industry-standard interfaces and file formats

Introduction to (ICC2) IC Compiler II

- IC Compiler II GUI

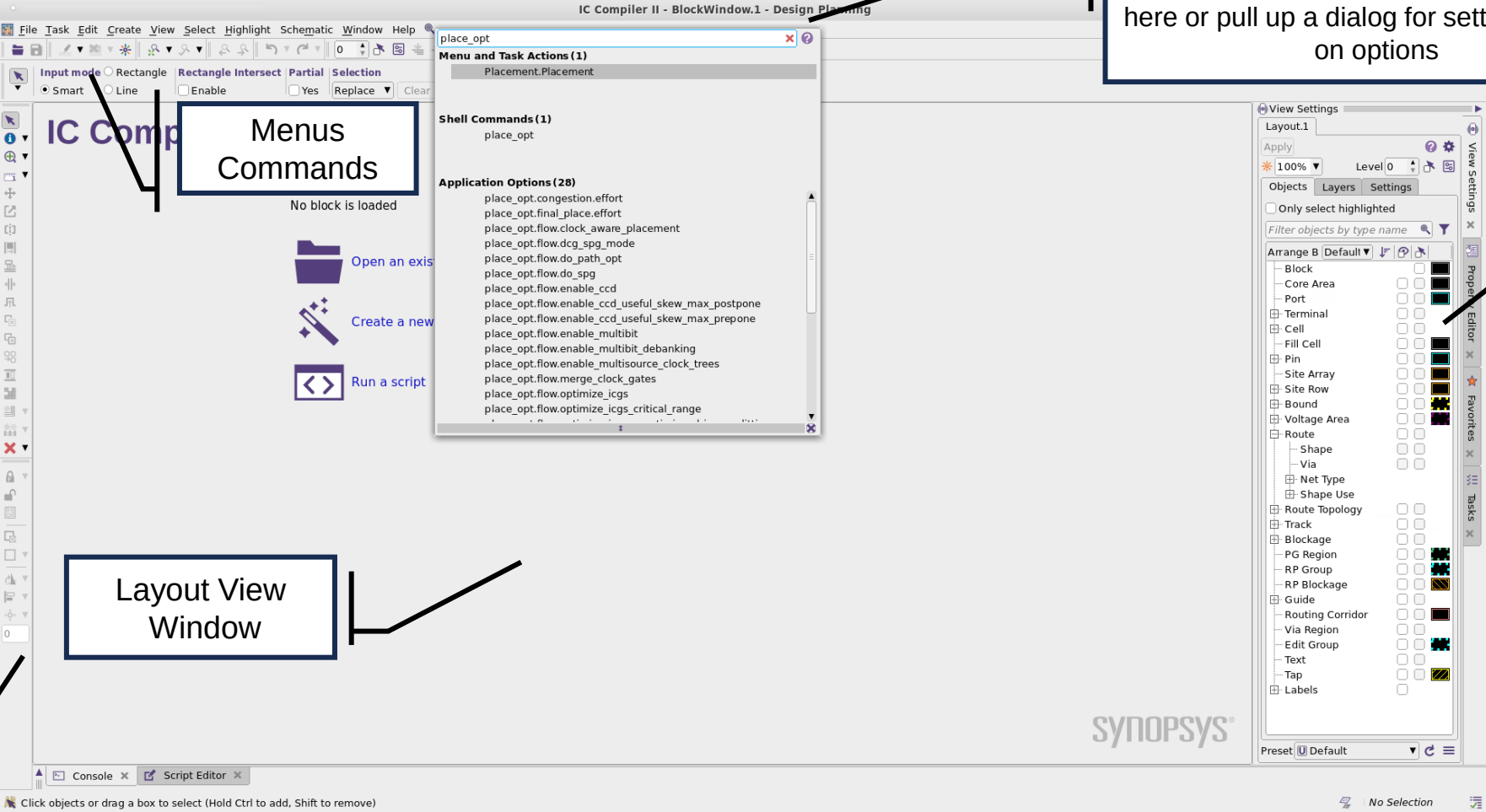
Command Search
Example: List menus, commands and options related to "place_opt"
You can also run the commands from right here or pull up a dialog for setting application options

Visibility and Selection control

**Menus
Commands**

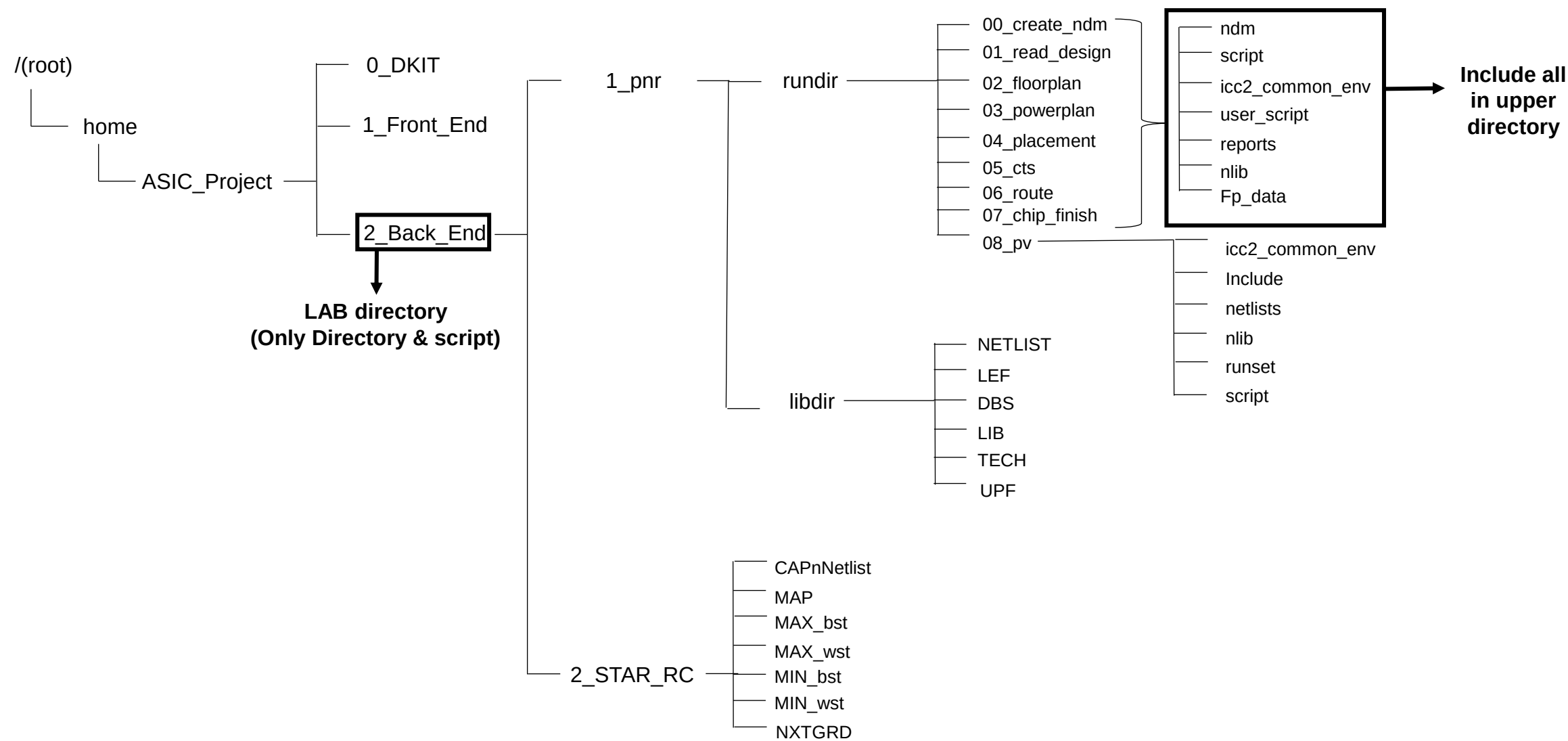
**Layout View
Window**

**Toolbar
Useful for
floorplan**



Directory structure

Directory Structure (Back End)



Backend Input files

Backend Input files : Type of Backend Input files

- Netlist
- SDC
- UPF
- Technology file
- TLU+
- Synopsys .db
- LIB
- LEF
- NDM
- IO information file

Backend Input files : Netlist

- What is Netlist

- Netlist is the combination of sequential elements and their logical connectivity
- Netlist contains standard cell instance (Name and Drive strength) and Macro, Memories(IP) instances
- Netlist also consists of Ports of Std cells and Macros, IP and Interconnection details
- Netlists provide instances, nodes, and attributes of the components involved
- Physical design is based on a Netlist which is the end result of the Synthesis process
- Synthesis converts the RTL design usually coded in VHDL or Verilog HDL to gate-level descriptions which the next set of ICC2 can read

- Example of Netlist

```
module adder (A, B, C, D)
    input A, B;
    output C, D;
    AND2 U1(.Y(C)), .A(A), .B(B)) ;
    XOR   U2(.Y(D)), .A(A), .B(B)) ;
endmodule
```

← Connectivity information between cells on the Netlist

Backend Input files : SDC

- What is SDC

- SDC stands for Synopsys Design Constraints
- SDC is a common format for constraining the design which is supported by almost all Synthesis, P&R and other tools
- SDC is a format used to specify the design intent, including the timing, power and area constraints for a design
- SDC constraints include Clock Definition (Time Period, Duty Cycle) and Timing Exceptions (False Paths, Asynchronous Paths)
- SDC is tcl based

- Example of SDC

```
create_clock -name {clock name} -period 100 -waveform { 40 50 } [get_ports {port_name}] ;# create clock / clock period 100ns<200b>

set_clock_uncertainty -setup 0.25 [get_clocks {clock name}]
set_clock_uncertainty -hold 0.12 [get_clocks {clock name}]

set_false_path -from [get_ports {ports name}] -to [get_clocks {clock list}] ;# set false path port or pin or end point of target cell<200b>
set_false_path -through [get_ports {ports name}] -to [get_clocks {clock list}]

set_input_delay -2 -clock [get_clocks {clock name}] -clock (rise or fall) ;# set input delay -2ns for input port relative to clock<200b>
set_output_delay -2 -clock [get_clocks {clock name}] -clock (rise or fall)

set_max_transition 0.3 [get_clocks {clock name}] -clock_path -(rise or fall) ;# set max transition time -0.3ns on clock path<200b>
```

Backend Input files : UPF

- What is UPF
 - UPF stands for Unified Power Format
 - UPF is a power specification file used in the design cycle to implement low-power design approaches
 - UPF indicates a designs power intent at a high level
 - UPF specification defines the names and number of power domains, their constituent elements in terms of HDL instances, ports, and nets for inferring or inserting corresponding cells and connecting power supplies and control signals
- Components of UPF
 - Design Scopes for the UPF Power Domains
 - Power Domain Interfaces and Power Domain Boundaries
 - Power Supply and Power Supply Networks
 - Primary Power and Primary Ground
 - Power States and Modes of Power Operations
 - Power Strategies

Backend Input files : Technology file

- What is Technology file

- Technology file defines units and design rules for Layers and Vias as per the technology
- Technology file has to load before loading other LEF files since it holds the Layer information for that particular technology
- Extension .tf is Synopsys technology file format

- Components of Technology file

- Name and number conventions of Layers and Vias
- Physical and Electrical parameters of Layers and Vias
- Direction/Type/Pitch/Width/Thickness/Resistance/Capacitance/Metal Density/Antenna Rule/Blockages/Design Rules/Grid Definition
- Site/Unit Tie definition

Backend Input files

- Technology file have information of basic unit, metal layer, vias and design rule etc
- Technology file refer to the below example

<pre> Technology { name = "saed32" date = "Dec 20 2012" dielectric = 3.45e-05 unitTimeName = "ns" timePrecision = 1000 unitLengthName = "micron" lengthPrecision = 1000 gridResolution = 1 unitVoltageName = "V" voltagePrecision = 1000000 unitCurrentName = "uA" currentPrecision = 1 unitPowerName = "pw" powerPrecision = 1000 unitResistanceName = "kohm" resistancePrecision = 10000 unitCapacitanceName = "ff" capacitancePrecision = 10000 unitInductanceName = "nh" inductancePrecision = 100 minBaselineTemperature = 25 nomBaselineTemperature = 25 maxBaselineTemperature = 25 minEdgeMode = 1 } </pre>	<pre> Layer "M2" { layerNumber = 13 maskName = "metal2" isDefaultLayer = 1 visible = 1 selectable = 1 blink = 0 color = "50" lineStyle = "solid" pattern = "twelldot" pitch = 0.152 defaultWidth = 0.056 minWidth = 0.056 minSpacing = 0.056 maxWidth = 5 sameNetMinSpacing = 0.056 protrusionTblDim = 4 protrusionFatThresholdTbl = (0.15,0.3,1.5,3) protrusionLengthLimitTbl = (0.1,0.15,0.7,1.5) protrusionMinWidthTbl = (0.06,0.07,0.15,0.3) fatTblDimension = 5 fatTblThreshold = (0.005,0.155,0.305,1.505,3.005) fatTblParallelLength = (0,0.15,0.3,1.5,3) fatTblSpacing = (0.056,0.064,0.12,0.6,0.7, 0.064,0.064,0.12,0.6,0.7, 0.12,0.12,0.12,0.6,0.7, 0.6,0.6,0.6,0.6,0.7, 0.7,0.7,0.7,0.7,0.7) minArea = 0.016 minEnclosedArea = 0.2 onWireTrack = 1 } </pre>	<pre> DesignRule { layer1 = "VIA1" layer2 = "VIA1" cut1TblSize = 3 cut2TblSize = 3 cut1NameTbl = (V1SQ, V1BAR, V1LG) cut2NameTbl = (V1SQ, V1BAR, V1LG) sameNetXMinSpacingTbl = (0.07, 0.08, 0.08, 0.08, 0.085, 0.085, 0.08, 0.085, 0.085) diffNetXMinSpacingTbl = (0.07, 0.08, 0.08, 0.08, 0.085, 0.085, 0.08, 0.085, 0.085) } </pre>
--	--	---

<Tech file basic units and information>

<Information about Layer>

<Design rules between Layers>

Backend Input files : Technology file

- Technology file have information of basic unit, metal layer, vias and design rule etc...
- Technology file refer to the below example

```
"VIA4" {
  layerNumber          = 18
  maskName             = "via4"
  isDefaultLayer       = 1
  visible              = 1
  selectable           = 1
  blink               = 0
  color                = "orange"
  lineStyle            = "solid"
  pattern              = "rectanglex"
  /*pitch              = 0.36*/
  defaultWidth         = 0.05
  minWidth             = 0.05
  minSpacing           = 0.07
  fatTblDimension      = 2
  fatTblThreshold      = (0.0, 0.16)
  fatTblDimension2     = 2
  fatTblThreshold2     = (0.0, 0.16)
  fatTblFatContactNumber = ("25,29", "27,31,25,29", "27,31,25,29", "28,32,27,31,25,29")
  fatTblFatContactMinCuts = (" 1, 1", " 1, 1, 2, 2", " 1, 1, 2, 2", " 1, 1, 2, 2, 4, 4")
  cutTblSize           = 3
  cutNameTbl           = (V4SQ, V4BAR, V4LG)
  cutWidthTbl          = (0.05, 0.05, 0.1)
  cutHeightTbl         = (0.05, 0.1, 0.1)
  sameSegAlignedUpperWireMaxSpacingThreshold = 0.08
  sameSegAlignedLowerWireMaxSpacingThreshold = 0.08
  sameSegAlignedCutMinSpacing = 0.07
}
```

<Information about Via>

DensityRule	{	
	layer	= "M1"
	windowSize	= 100
	minDensity	= 1
	maxDensity	= 60
	}	
Color	6 {	
	name	= "6"
	rgbDefined	= 1
	redIntensity	= 0
	greenIntensity	= 80
	blueIntensity	= 190
	}	
Tile	"Unit" {	
	width	= 0.152
	height	= 1.672
	}	

<Density Rule/Color/Unit Tile/height Tile information>

Backend Input files : TLU+ file

- What is TLU+
 - TLU+ file is a binary table format that stores the RC Coefficients
 - TLU+ models enable accurate RC extraction results by including the effect of width, space, density and temperature on the resistance coefficients
 - The map file matches the layer and via names in the Milkyway technology file with the names in the ITF (interconnect Technology Format) file.
 - R, C parasitic of metal per unit length (these used for calculation Net delays)
 - If TLU+ files are not given, then these are extracted from .ITF file
 - We need Max TLU+, Min TLU+ and Map file for loading TLU+ files
 - Map file maps the .ITP file and .tf(technology) file of the layer and via names
 - TLU+ files, like ndm files, cannot be opened with gvim or vim to view the contents (nonhuman readable file)

Backend Input files : Synopsys db & lib

- What is Synopsys db & lib

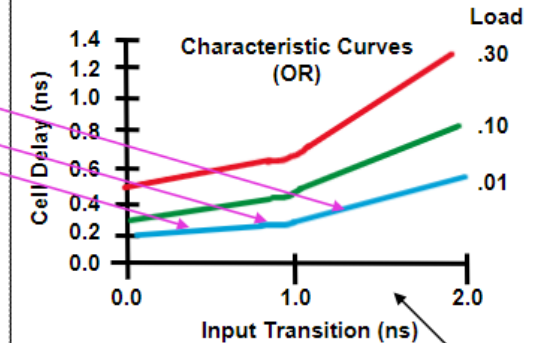
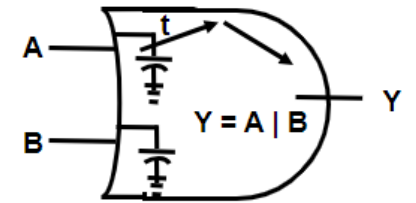
- .lib files are cell logical View/Timing Library
- Types of this file include (std. Cell lib, Macro lib, IO lib) files
- This files contains Gate Delay (function of input transition time and output capacitance)
- .db file is a conversion format in which .lib is converted to create NDM

- Components of Synopsys lib

- Cell Type and Functionality
- Delay Models(WLD/NLDM/CCS)
- Pin/Cell Timings and design rules
- PVT Conditions
- Power Details (Leackage and Dynamic)

❖ Example of a cell description in .lib file format

```
cell ( OR2_4X ){  
  area : 8.000 ;  
  pin ( Y ) {  
    direction : output ;  
    timing ( ) {  
      related_pin : "A" ;  
      timing_sense : positive_unate ;  
      rise_propagation (drive_3_table_1) {  
        values ( "0.2616, 0.2711, 0.2831, ..." )  
      }  
      rise_transition (drive_3_table_2) {  
        values ( "0.0223, 0.254, ..." )  
      }  
    }  
    ...  
  }  
  function : "(A | B)" ;  
  max_capacitance : 1.14810 ;  
  min_capacitance : 0.00220 ;  
  ...  
  pin ( A ) {  
    direction : input ;  
    capacitance : 0.012000 ;  
    ...  
  }  
}
```



Propagation & Transition tables are "characterized" for a specific PVT corner, e.g. slowest or worst-case

During "compile_ultra" optimization, logic or buffering DRCs have higher priority than timing, by default

Backend Input files : Delay Models & calculation

- What is Delay calculation
 - Delay calculation is needed because of complex input capacitance, voltage Drop, High impedance nets etc
 - Delay calculation parameter data are stored as Lookup-Table format
- Interconnect Delay Models(WLD) & Cell Delay Models(NLDM/CCS)

Interconnect Delay Models	Cell Delay Models	
WLD(Wire load Delay)	NLDM(Non_Linear Delay Model)	CCS(Composite Current Source Delay Model)
• Delays are estimated based on the number of fanout of the cell driving the net • Values of unit resistance R and unit capacitance C are given in .tf file • Fanout versus net length is tabulated in WLMS • Once the net length is known delay can be calculated	• Less accurate • Less run time • Intermediate values are interpolated • Assumes load is purely capacitive • Variation may range anywhere from 5~10% • Transition time are functions of input slew and output load	• More accurate • More run time • Extra setup required for characterization • CCS libraries are huge in size

Backend Input files : LEF file

- What is LEF file
 - LEF file stands for Library Exchange Format
 - LEF file provides technology information, such as metal and via layer information and via generate rules
 - LEF file contains all the physical information for the design
 - LEF file is in ASCII format so this file is human-readable file
 - Types of this file include (Std, Cell LEF, Macro LEF, IO LEF) files
- Components of LEF file
 - Cell Name, Shape, Size, Orientation & Class
 - Port/Pin Name, Direction and Layout Geometries
 - Obstruction/Blockages
 - Antenna Diff, Area

- Example of LEF file

```
VERSION 5.4 ;
NAMECASESENSITIVE ON ;
BUSBITCHARS " < " ;
DIVIDERCHAR " / " ;

MACRO TNBUFFX2_RVT
  CLASS CORE ;
  SOURCE USER ;
  ORIGIN 0 0 ;
  SIZE 2.128 BY 1.672 ;
  SYMMETRY X Y R90 ;
  SITE unit ;

  PIN EN
    DIRECTION INPUT ;
    USE SIGNAL ;
    PORT
      LAYER M1 ;
      RECT 0.3240 1.3730 0.7360 1.4230 ;
      RECT 0.5530 1.4230 0.6630 1.5750 ;
    END
    PORT
      LAYER CO ;
      RECT 0.6630 1.3770 0.7050 1.4190 ;
      RECT 0.3590 1.3770 0.4010 1.4190 ;
    END
    ANTENNAGATEAREA 0.0264 ;
  END EN
```

Backend Input files : NDM & IO information file

- What is NDM

- NDM stands for New Data Model in ICC2
- NDM file is .tf - Vendor Technology file.
- NDM file is a Synopsys library format that stores all of circuit files from synthesis through place and route all the way to signoff
- NDM file contains technology-specific information such as the names, characteristics (physical and electrical) for each metal layer, and design rules (These information are required to route a design)
- NDM file cannot be opened with gvim to view the contents (nonhuman readable file)

- What is IO information file

- IO information file is a file for placing a pad, pin, etc
- IO information file contains edge and order for io placement
- IO information file contains Pin/Pad locations
- Extension .tdf, .io are common formats

Cells & Layer

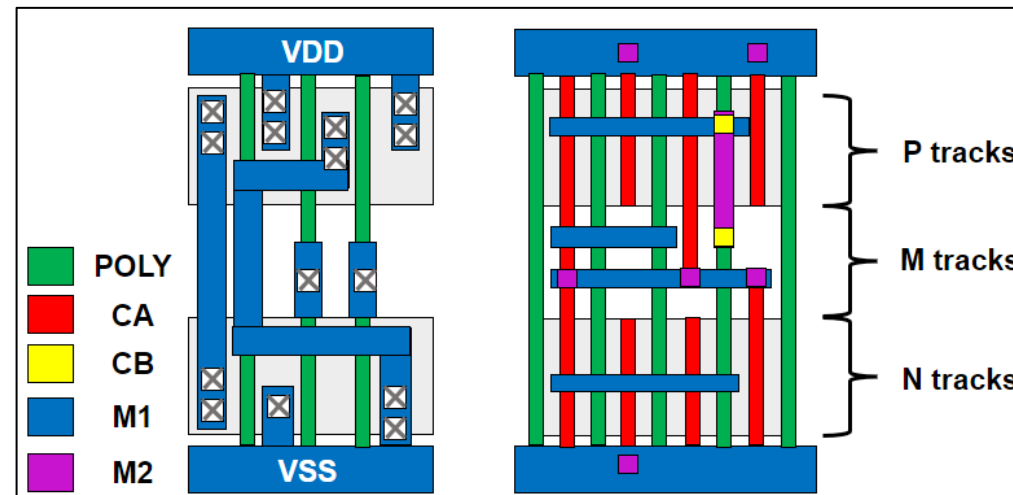
Cells & Layer : Standard cell

- What is standard cells

- Collection of well defined and pre-characterized logic cell such as AND, OR, INVERT, flip-flops, latches, and buffers
- Physical cells and a set of library files required by Place and Route (P&R) tool for automatic placement and routing
- Cell-based methodology makes designer to focus on the implementation physical aspects

- Types of Standard cells

- Combinational (AND, OR, NAND, NOR, AOI, OAI, OA, AO, MUX)
- Arithmetic (XOR, full-adder, half-adder)
- Sequential (latches, clock gates, D-type flip/flops with any optional combination of scan input, set and reset)
- Miscellaneous (ICG Cells, Well Taps, Tie Cells, End Caps, Decaps, Filler Cells, Spare Cells, Delay Cells, Antenna Diode, ESD diode s)



Example of standard cell

Synopsys University education program

Cells & Layer : HVT RVT LVT & PVT Corners

- What is (HVT RVT LVT) cells

HVT	High Threshold Voltage causes less power consumption and timing to switch is not optimized power critical functions
RVT	Standard Threshold Voltage offers trade-off between HVT and LVT moderate delay and moderate power consumption
LVT	Low Threshold Voltage causes more power consumption and switching timing is optimized time critical functions

- PVT Corners

Process	The 'P' in PVT corners in ASIC stands for Process Process variations are caused by changes in manufacturing conditions such as temperature, pressure, and dopant concentrations.
Voltage	Any semiconductor chip's operating voltage is supplied from the outside Voltage variation is caused by changes in the operating environment of the chip
Temperature	Temperature outside affects the timing Temperature variation is caused by changes in the operating environment of the chip

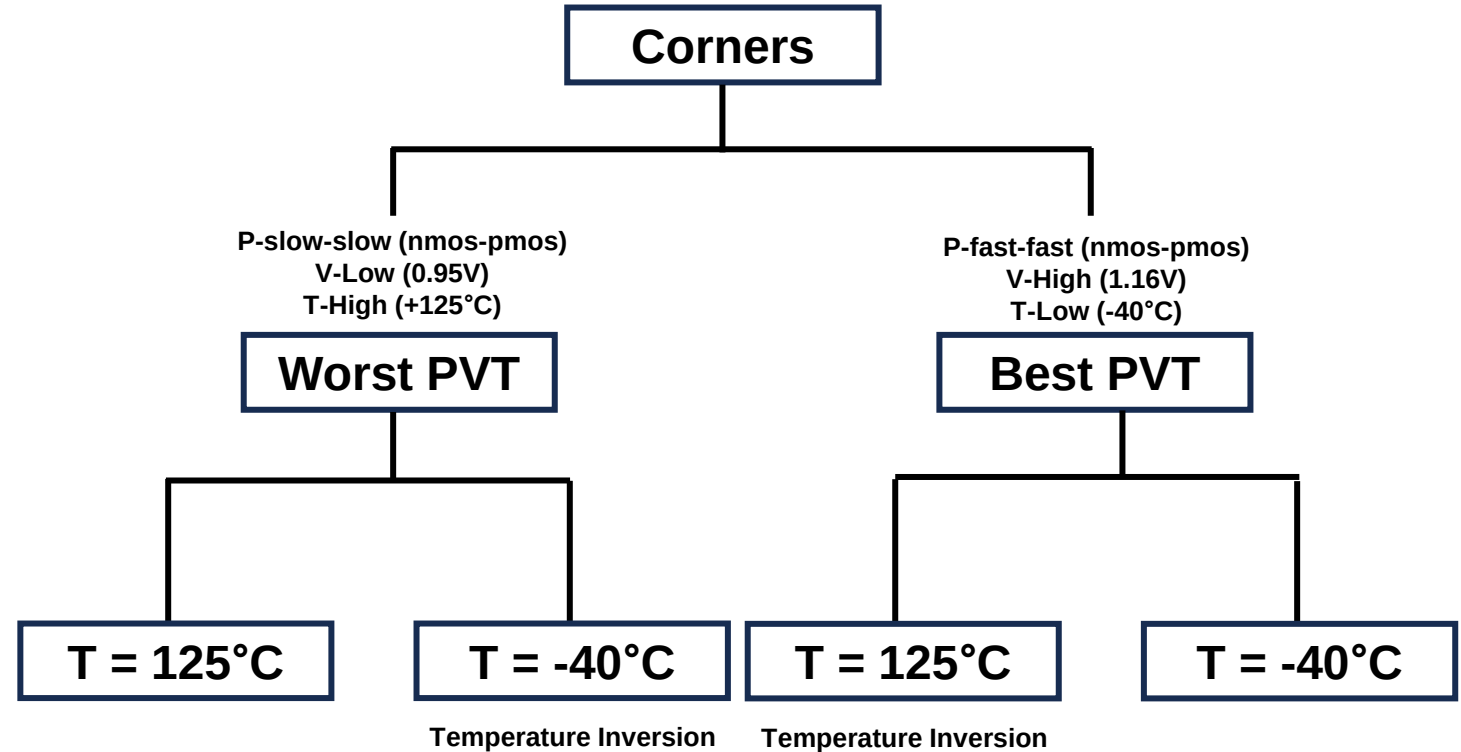
Cells & Layer : PVT Corners

- **28/32nm DesignKit operating conditions**

- **PVT conditions for 28/32nm DK**

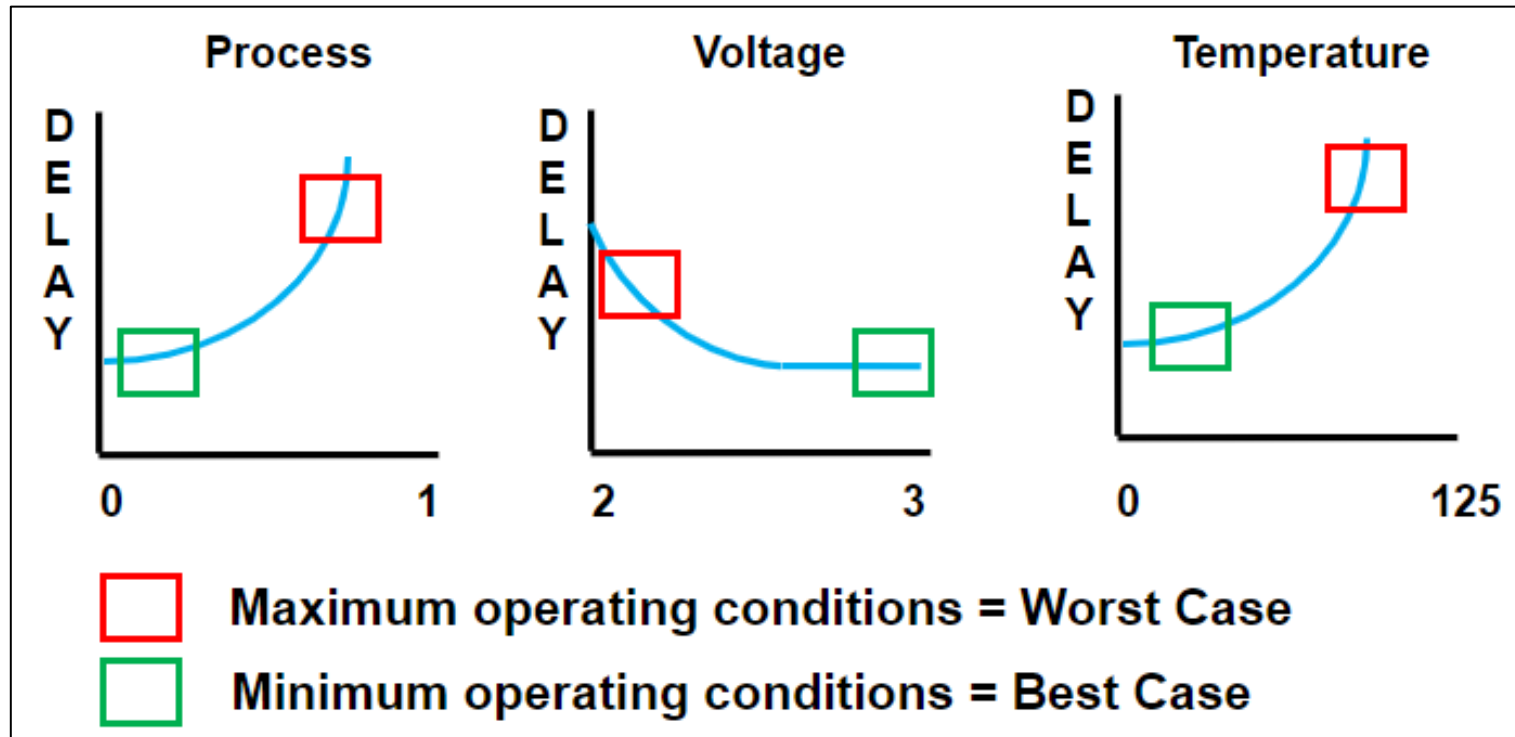
(P: Process V: Voltage T: Temperature)

- TYP Corner : TT/1.05V/25C
- WST Corner: SS/0.95V/125C
- WST temp inversion Corner : SS/0.95V/-40C
- BST Corner : FF/1.16V/-40C
- BST temp inversion Corner : FF/1.16V/125C



Cells & Layer : PVT Corners

- PVT Corners



- The Best and Worst design corners are defined as follows
 - Best case : fast process, highest voltage and lowest temperature
 - Worst case : slow process, lowest voltage and highest temperature

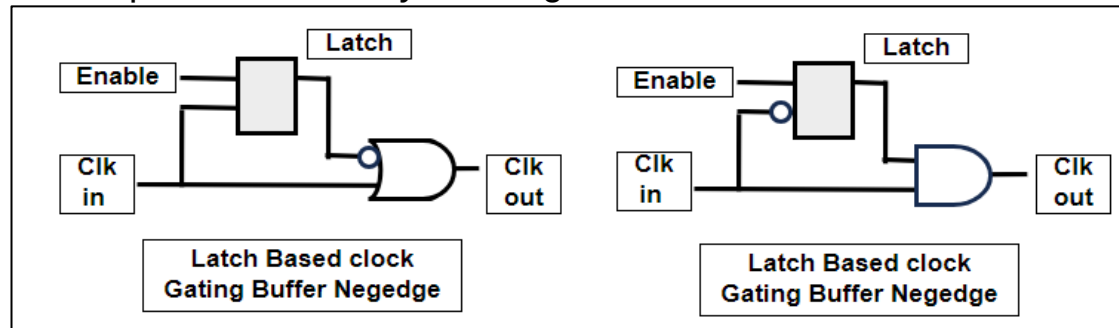
Cells & Layer : ICG(Integrated Clock Gating) cell

- What is ICG cells

- ICG cell is a standard cell used in one of low power techniques in ASIC design
- It is used to reduce the dynamic power in SoC Design
- It is used to stop the clock signal propagation to a big group of logic cells when the group is not required to operate
- The ICG cell stops the clock propagation through it when a low clock enable signal is applied to it, a phenomenon known as clock gating

- Types of Clock Gating cells

- Latch Based Clock Gating Buffer for Neg-edge
 - The circuit employs a latch and OR gate with one input inverted
 - The output clock is always clock gated low when Enable is low
- Latch Based Clock Gating Buffer for Pos-edge
 - The circuit employs a latch with inverted clock input and a AND gate
 - The output clock is always clock gated HIGH when Enable is low



Cells & Layer : Physical Cells

- What is Physical cells
 - Physical cells in ASIC are used to meet some DRC rules and for design protection
 - Physical cells don't have any logic pins
- Types of Physical cells
 - ENDCAP cell (Boundary cell)
 - TAP cell
 - DECAP cell
 - SPARE cell
 - TIE cell
 - Delay cell
 - FILLER cell
 - Switch cell (Power gating)
 - Isolation cell

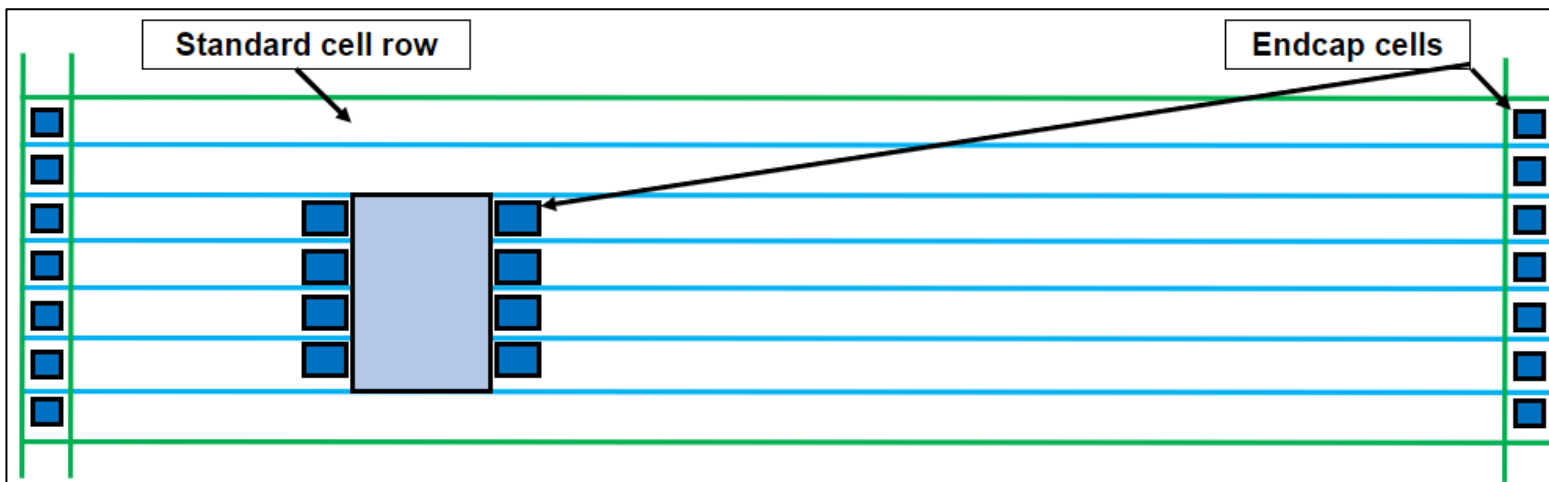
Cells & Layer : Physical cell

- ENDCAP cell (Boundary cell)

- What is ENDCAP cell

- End-cap cells are preplaced physical-only cells required to meet certain design rules and placed at the ends of the site rows by satisfying well tie-off requirements for the core rows
 - End-cap cells are positioned in the design to guard against manufacturing damage to a regular cells gate that is located close to the boundary
 - End-cap cells do not have any signal connectivity and they connect only to the power and ground rails
 - Each end of the core row, left and right, can have only one end cap cell specified

- Example of ENDCAP cell



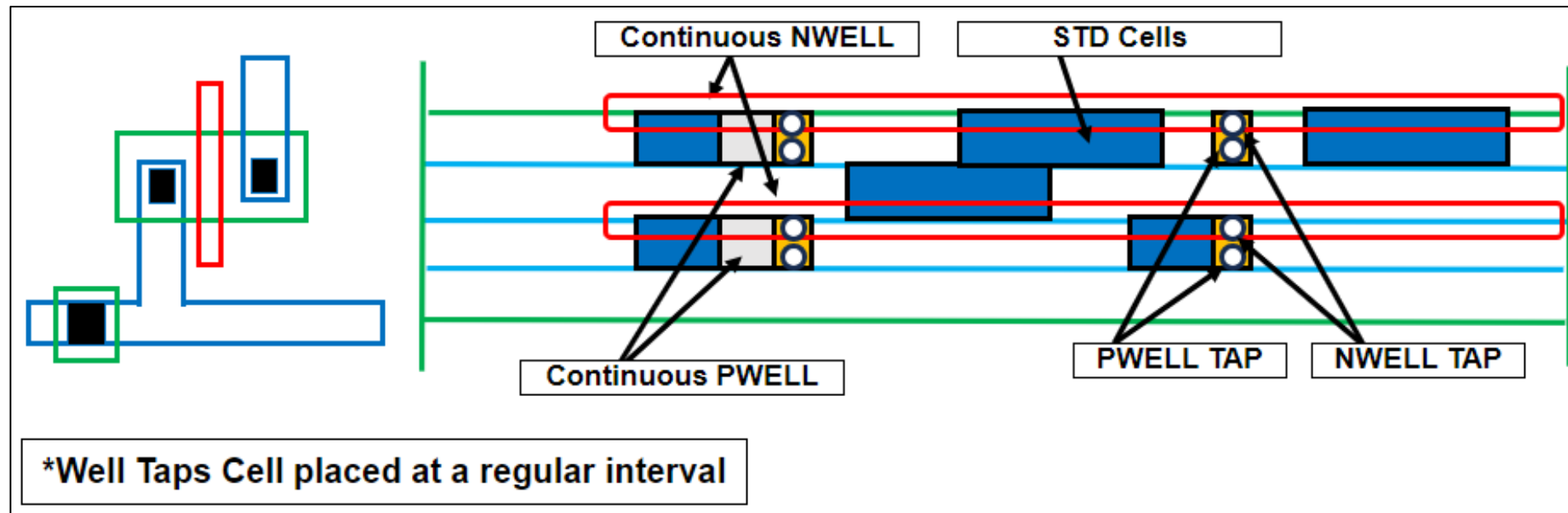
Cells & Layer : Physical cell

- TAP cell

- What is TAP cell

- TAP cell is a special no logic cell with a well tie, substrate tie, or both
- TAP cells reduce resistance between VDD/GND to wells of the Substrate
- TAP Cells are do not have any signal connectivity and they usually placed on the Power Rails of the Standard Cells (VDD/GND)
- TAP cells are used to prevent latch-up in CMOS

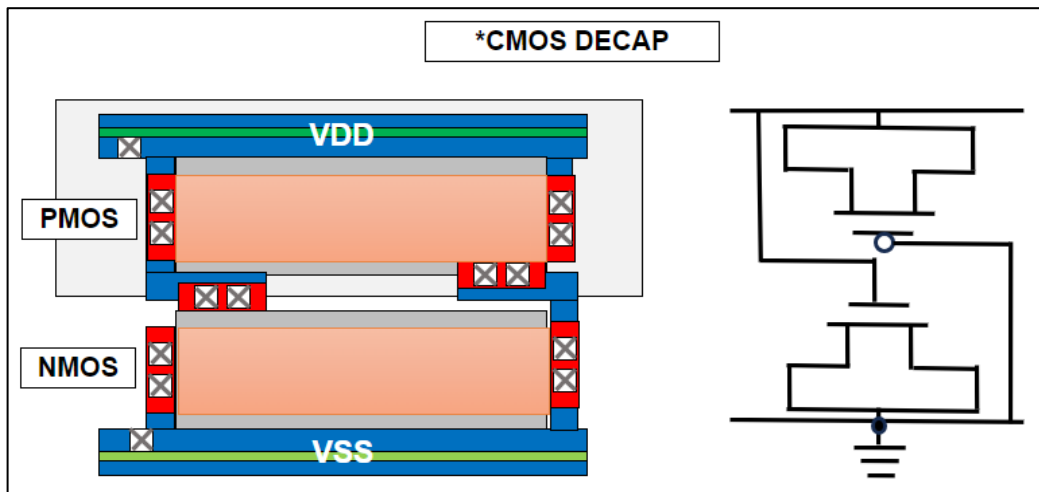
- Example of TAP cell



Cells & Layer : Physical cell

- Decap cell

- What is Decap cell
 - Decap cells are on-chip decoupling capacitors that are attached to the power mesh to decrease noise effects
 - Decap cells are typically poly gate transistors where source and drain are connected to the ground rail and the gate is connected to the power rail
 - Decap cells help to smoothen out the Glitches and Ground bounce 3% ~ 8% of the core physical area is required for Decaps referred as Decap density and it is important to place only the necessary amount of Decaps since they normally come with a quite serious down-side as they are leaky devices
 - Decap cells are placed generally after finish powerplan, and before placement
- Example of Decap cell



Cells & Layer : Latch up & Glitch

- What is Latch up

- Latch-up is a short circuit/low impedance channel generated between the power and ground rails of a MOSFET circuit resulting in high current leading to IC damage
- Latch-up is caused by the interaction of parasitic PNP and NPN transistors (BJT's)
- Latch-up creates a positive feedback loop by short-circuiting the power and ground rails, resulting in excessive current and it causes damage to the device

- What is Glitch

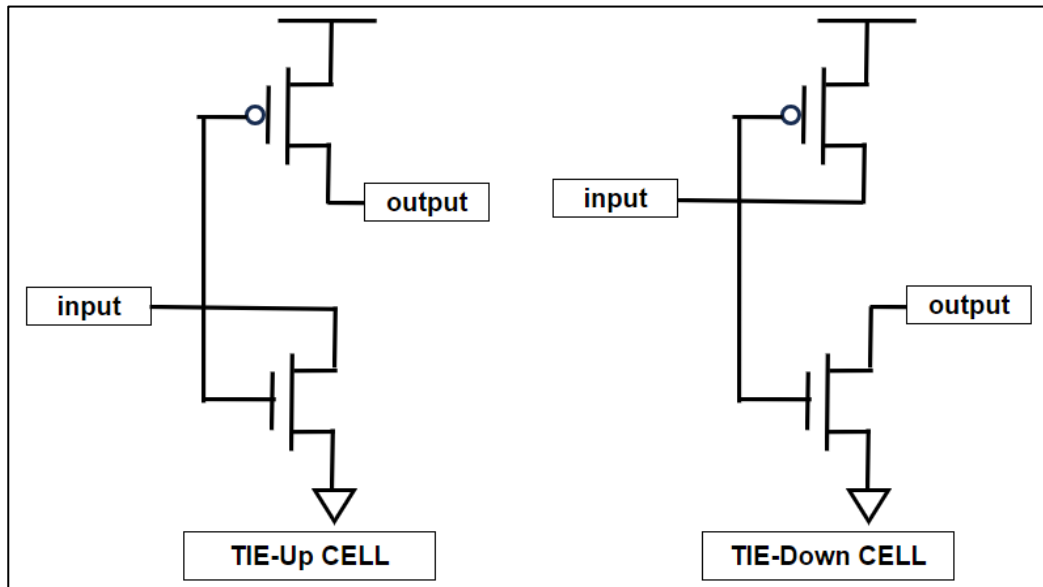
- Glitch is any unwanted pulse at the output of a combinational gate
- In electronics design, glitch refers to unnecessary signal transitions in a combinational circuit, while glitch power refers to the power consumed by glitches
- Since glitches can cause potential electro-mechanical (EM) and IR drop issues (even in the power grid), they should be suppressed by the designers or via implementation or ECO tools

Cells & Layer : Physical cell

- TIE cell

- What is TIE cell
 - A tie cell is a standard cell designed to provide a high or low signal to the input of any logic gate
 - Tie cells are used to connect the Gate of the transistor to either Power or Ground
 - Tie High is a Power supply cell, because the cells which require VDD, comes and connect to TIE High, while the cells which wants VS connects itself to TIE Low

- Example of TIE cell

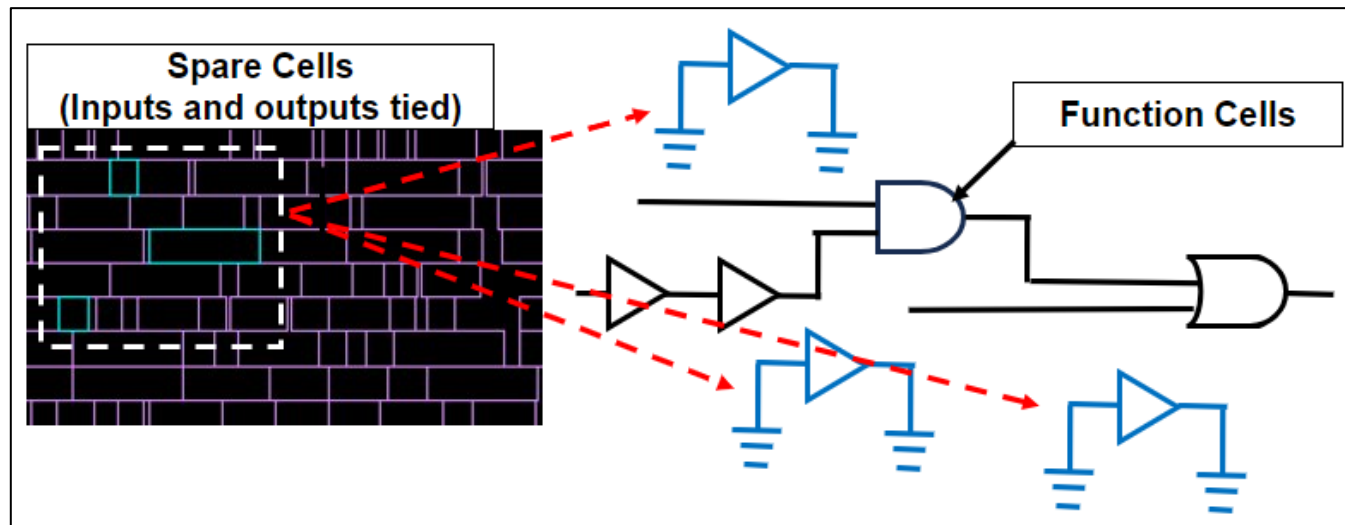


Cells & Layer : Physical cell

- Spare cell

- What is Spare cell
 - Pre-placed inactive gates in the empty areas of a design (or even in the crowded areas) before tape-out
 - Spare cells are collection of gates coming in different sizes for doing small functional ECO and connect them with a metal-only ECO
 - Spare cells are not timing-critical and are used to fix bugs or add features after the design is tapped out
 - Provides new functions on a design which exhibits post-production problems
 - Metal and a contact layer only need to change
 - Spare cells do not perform any logical operation in the design and act as a filler cell only

- Example of Spare cell



Cells & Layer : Physical cell

- Delay cell

- What is Delay cell

- Delay cell insertion is the conventional way to fix hold time violation tends to penalized in area percentage increment
 - Lesser number of delay cells are required for hold time fixing as compared to buffers but it will have area much greater than normal buffers
 - Increasing gate width reduces gate capacitance hence reduces delay, but results in higher leakage
 - Depending on the delay of the cell, pair of inverter and pass transistor can be repeated multiple times

- What is Delay

- $\text{Delay} = \text{Cell Delay} + \text{Net Delay}$

- What is Cell Delay

- Cell delay is the timing delay between input and output pin of a cell.
 - It is the time required to charge, discharge the load capacitance

- What is Net Delay

- Net means a wire, and Net Delay means a delay that occurs as it passes through the wire.

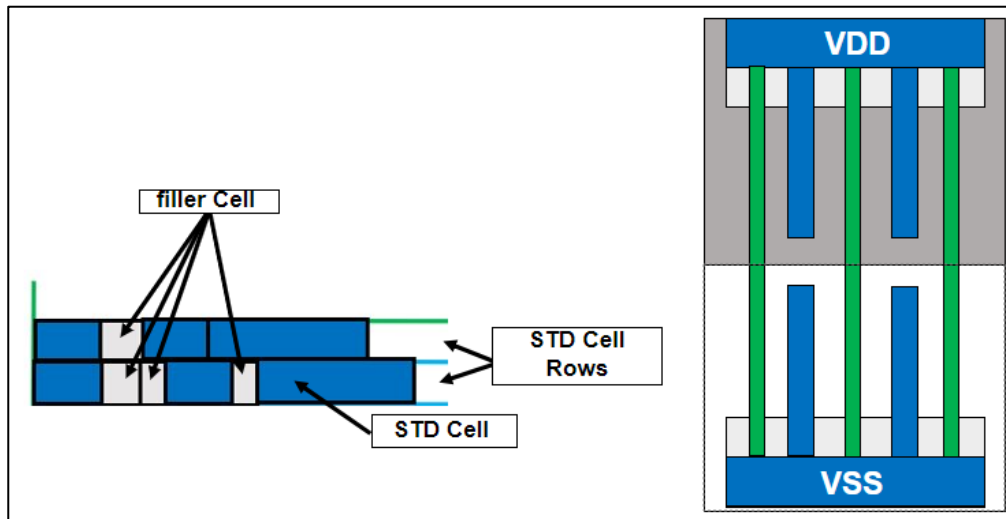
Cells & Layer : Physical cell

- Filler cell

- What is Filler cell

- Filler cell is a physical only cells which provide N-Well continuity and avoid N-Well spacing DRC at PV
 - Filler cells are inserting for density rules, to meet Core Utilization targets and to avoid sagging of layer
 - Filler cells are inserting at the last stage of Chip finish
 - Filler cells also used to make up the poly density
 - Filler cells are useful for ECO step

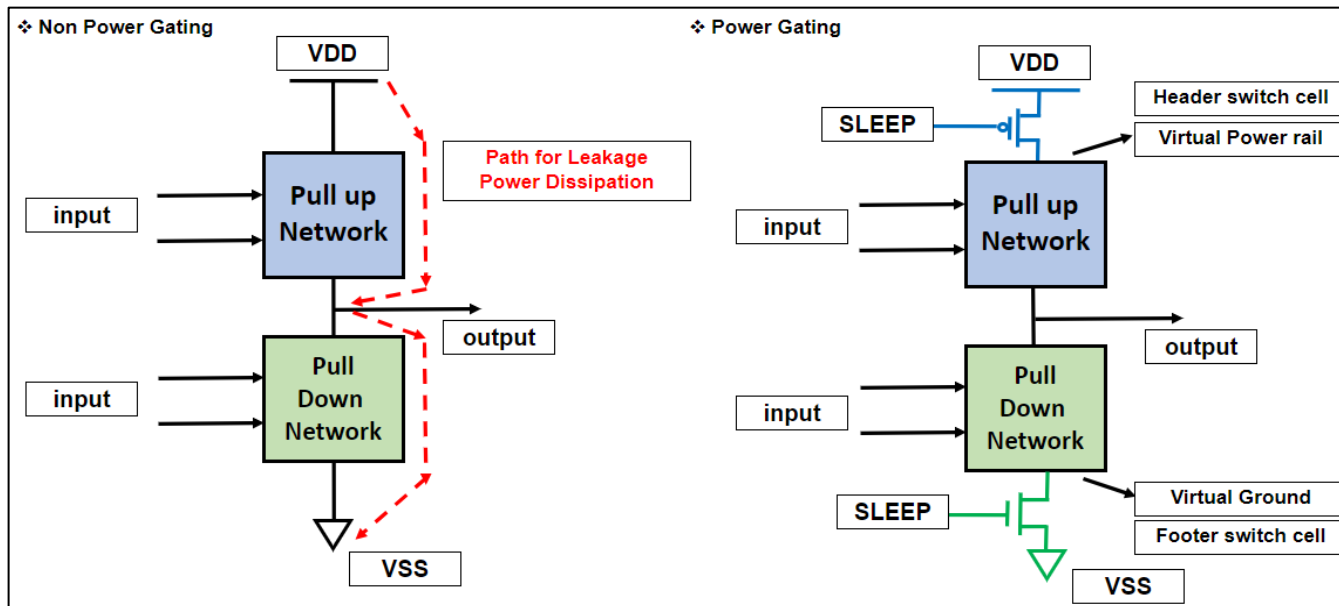
- Example of Filler cell



Cells & Layer : Power Gating

- What is power Gating

- Power gating is a technique used in ASIC design to reduce the overall leakage power of the chip
- This is accomplished by adding a switch either to power or ground supply (power switch cell)
- Powering OFF a design block is the most beneficial technique of all the low power techniques because you dissipate near zero power
- VDD is gated the power switch is called the “header” switch. Similarly if VSS is gated we call it as “footer” switch. To gate power we use PMOS transistor where as to gate ground we use NMOS transistor
- Single switch is not sufficient to power the whole design. Hence many parallel switches are implemented to achieve low voltage ramp up time and Decrease other IR drop issues



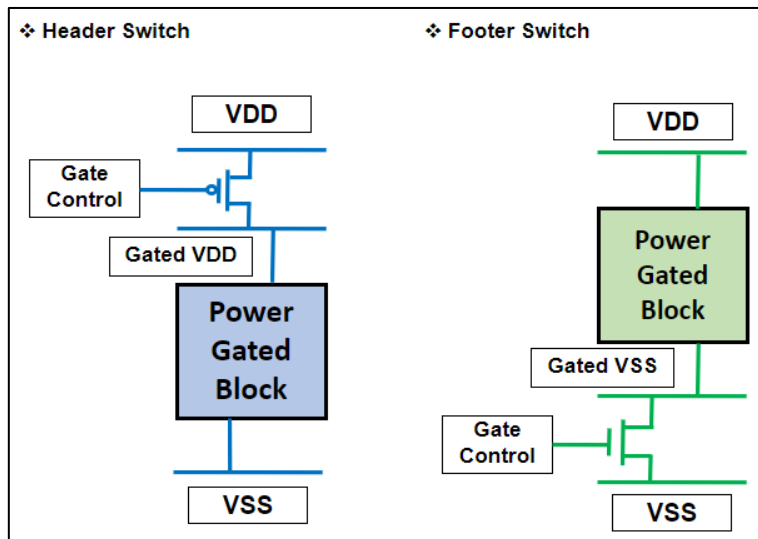
Cells & Layer : Physical cell

- Switch cell

- What is Switch cell

- Switch cell is basic element which is used in power gating technique to shutting down the power for a portion of the design
- Switch cells are defined in UPF
- Switch cells in ASIC are cells that can gate the power supply of block of cells when they are not required
- Switch cells are also called power gating switches
- Switch cells use high-VT transistors as header switches, which have very high threshold voltage when device is OFF and very low threshold voltage when device is ON. This reduces the leakage power of the gated block

- Example of Switch cell



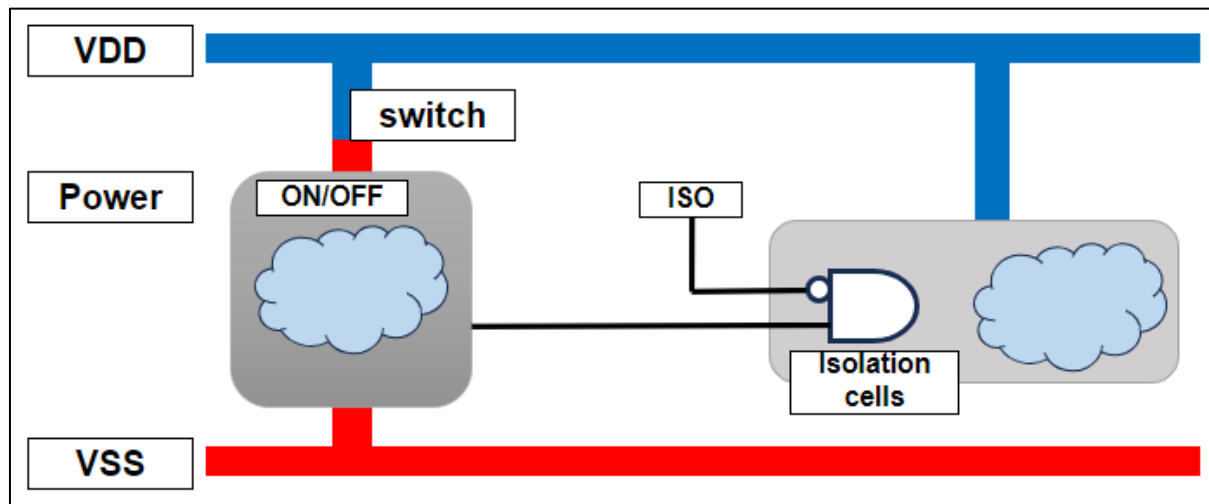
Cells & Layer : Physical cell

- Isolation cell

- What is Isolation cell

- Isolation cells are extra cells introduced by synthesis tools to isolate buses/wires crossing from a circuit's power-gated domain to its always-on domain
 - Isolation cells are often located between two power domains and connect domains that are shut off to domains that are still switched on
 - Isolation cells are employed to separate domains
 - Isolation cells are defined in UPF

- Example of Isolation cell



Cells & Layer : IO PAD

- What is IO PAD

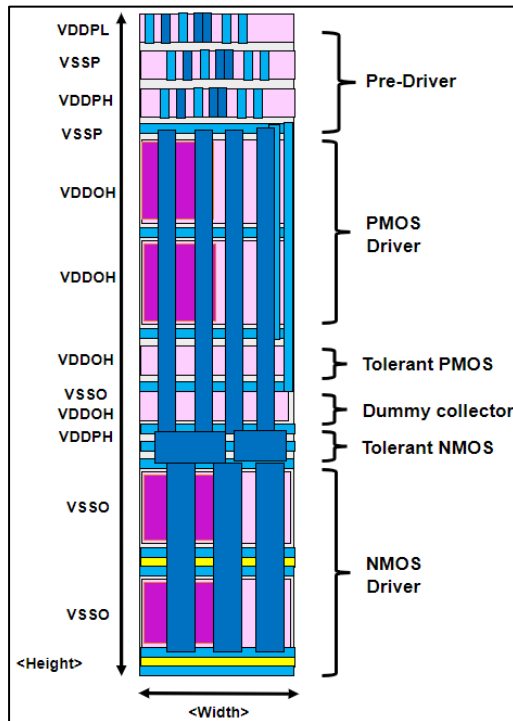
- IO Pads are intermediate structures connecting internal signals from the core of the integrated circuit to the external pins of the chip package
- Typically IO pads are organized into a rectangular Pad Frame
- The IO pads are spaced with a Pad Pitch
- Pads will have pins on all metal layers used in design for easy access while routing the design (Number of layers depends on technology)
- Multiple Power Pads are often used to reduce the power
- Pads consists of some logic cells like level shifters and buffers which will control the voltages of input and output signals and to increase/ decrease drive strength (GPIO: General Purpose Input Output)

- Types of PADS

- Signal Pad
- Power Pad (Core & IO power)
- Corner Cell
- IO Filler Cell
- Bond Pad

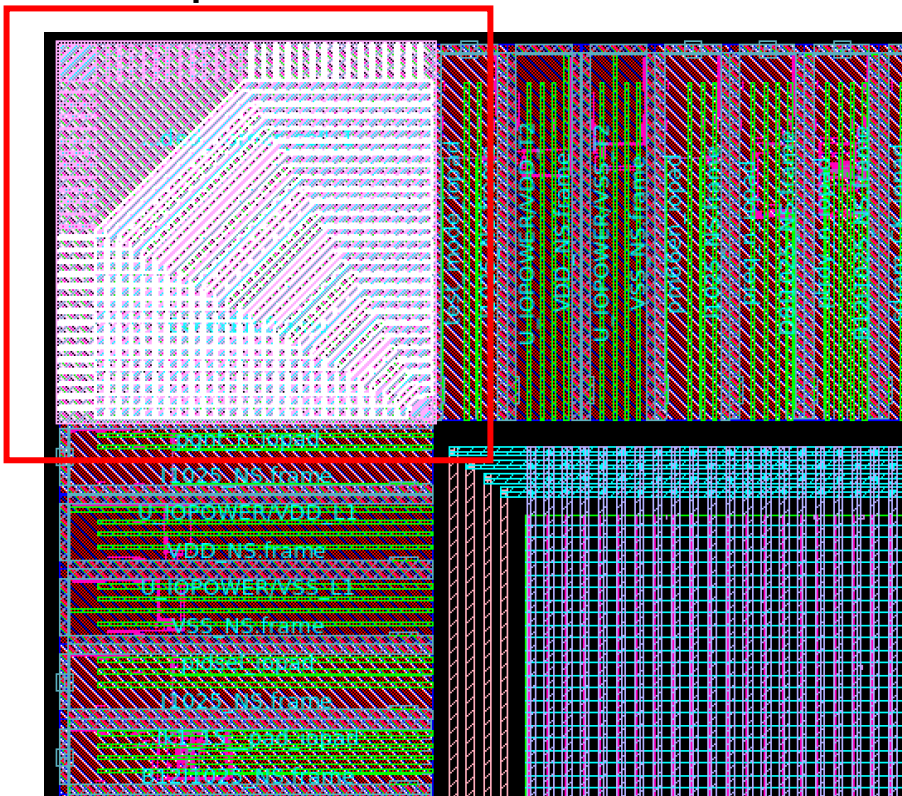
Cells & Layer : Signal & Power PAD

- What is Signal & Power PAD
 - Power Pads are PAD that supplies power to the chip
 - Signal Pads are intermediate structures connecting internal signals from the core of the integrated circuit to the external pins of the chip package
 - IO pads are organized into a rectangular Pad Frame
- Example of Signal & Power PAD



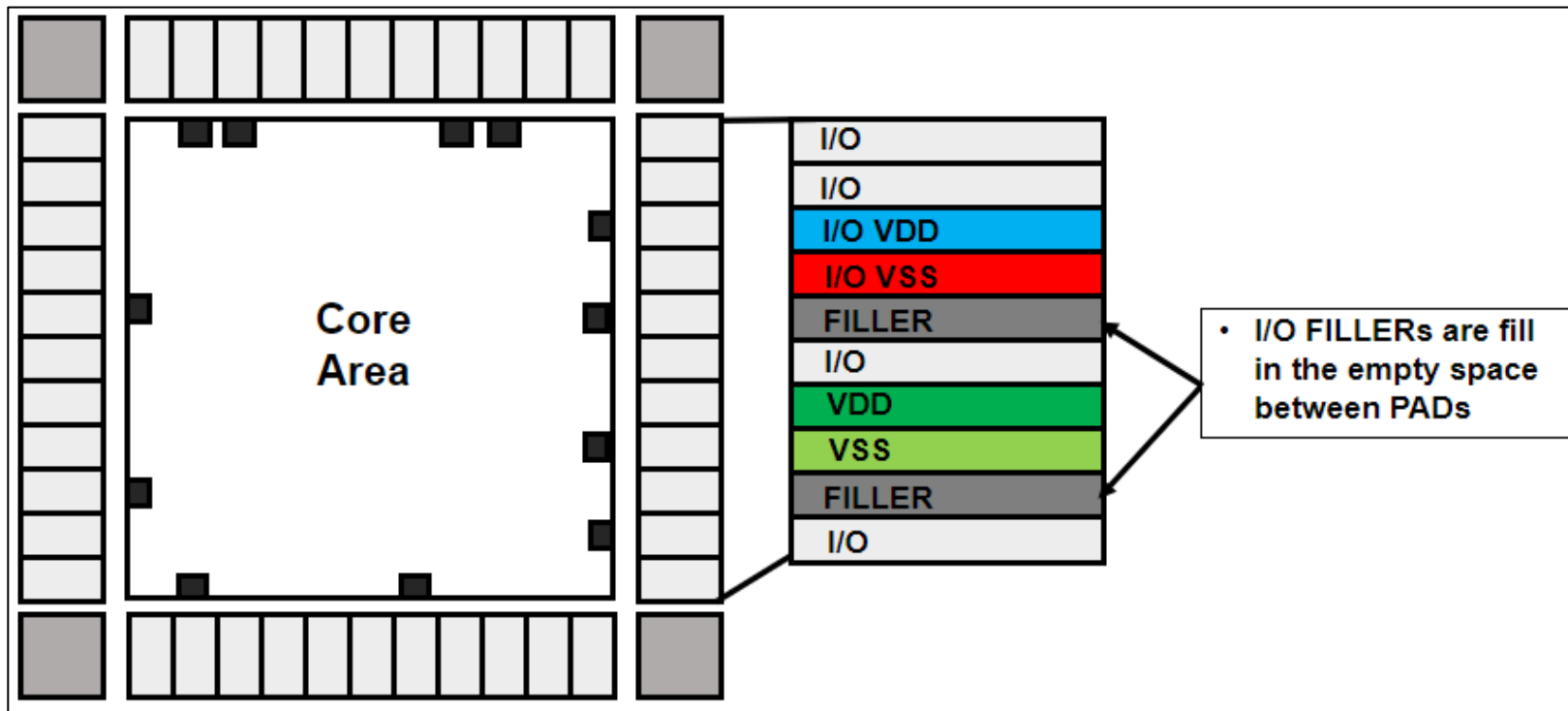
Cells & Layer : Corner Cell

- What is Corner Cell
 - Corner cells contains only connections in all metal layers defined in technology
 - These cells are used only for IO Ring continuity and chip metal density on corners and to maintain yield
- Example of Corner Cell



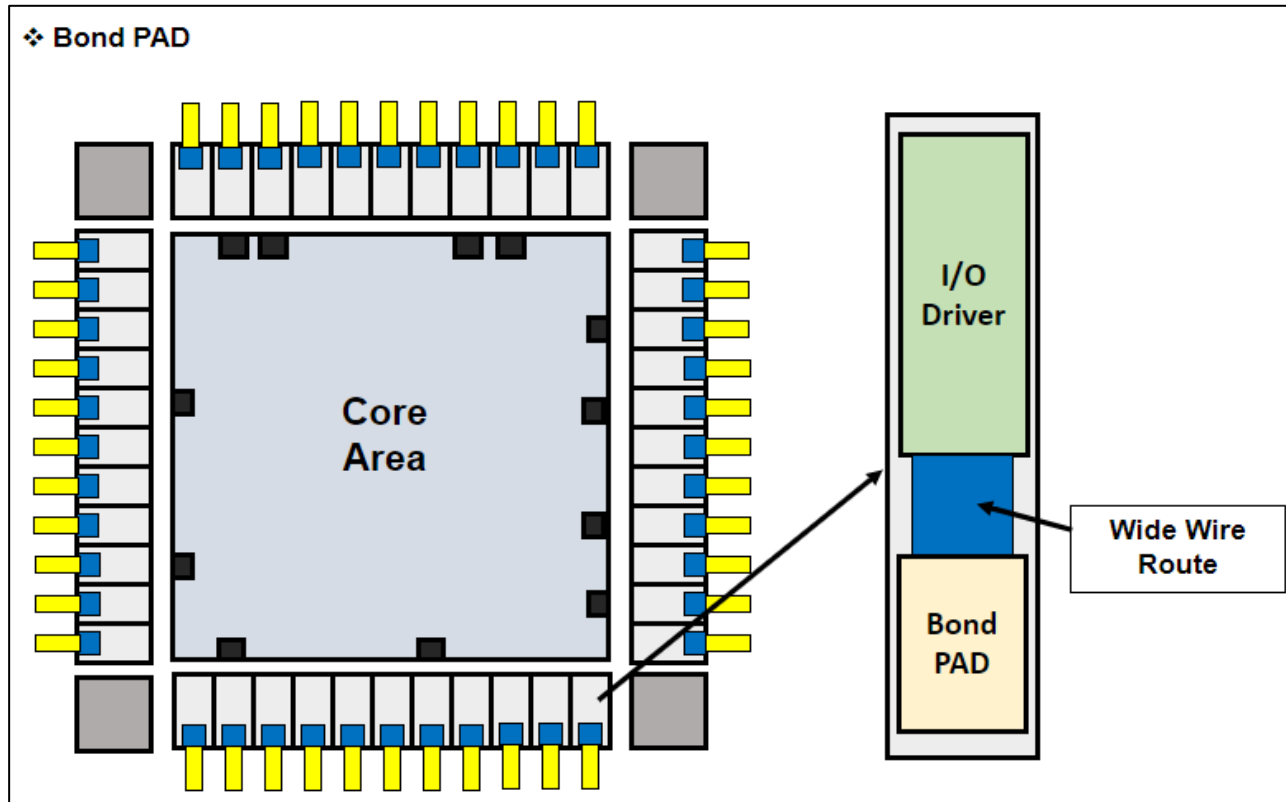
Cells & Layer : IO Filler cell

- What is IO Filler Cell
 - IO Filler Cells contains only the geometrical information of the Power Rings in all metal layers
 - Continuity of Power Rings which is responsible for uniform distribution of power
 - Electrostatic Discharge protection
- Example of IO Filler Cell



Cells & Layer : Bond PAD

- What is Bond PAD
 - Bond PADS are area to which the bond wire is soldered
 - The wires goes from the bond pad to a chip pin
- Example of Bond PAD

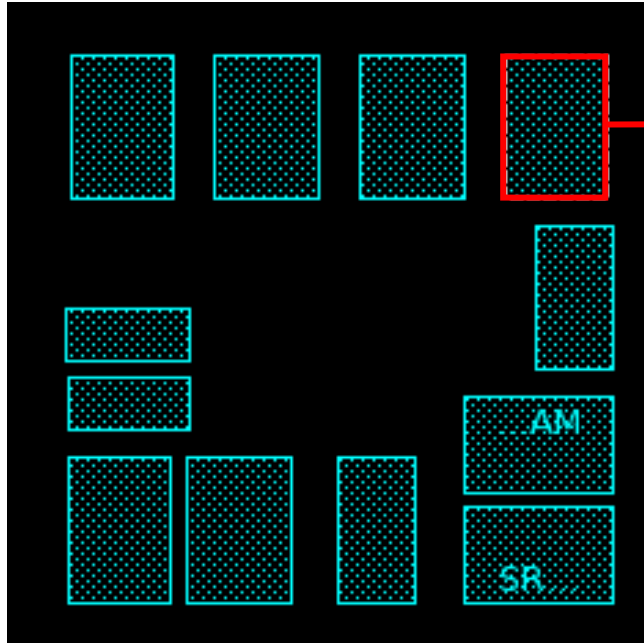


Cells & Layer : Memory Cells (Macro)

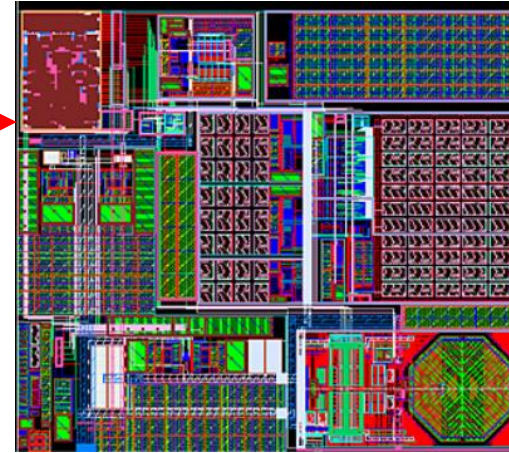
- What is Memory Cells (Macro)
 - Memory Cells are the Macro Cells
 - This Cells have been designed by some other Analog design team(full custom), which can be used in the floorplan stage
 - During P&R flow, the hard macro type is mainly used
- Type of Macros
 - Hard Macros
 - Soft Macros

Cells & Layer : Memory Cells (Macro)

- Example of Macro



<ICC2 Hard Macro>



<Internal pattern of Hard Macro>

Hard macros used in ICC2 cannot see the internal pattern
Because we use the one designed by the analog team and converted into a LEF file
If all the internal patterns are included, the db size becomes very large, and thus the run time between flows may be very long

Cells & Layer : Memory Cells (Macro)

- What is Hard Macro
 - Hard Macros are defined in LEF or GDS files
 - Hard Macro always comes in terms of hardware IPs
 - Hard macros are very similar to block level design and optimized for PPA(Power, Performance, Area)
 - While placing hard macros during floorplan, it can only move, rotate, flip, but can't change anything inside it
- What is Soft Macro
 - Soft Macros are defined in Synthesizable RTL
 - RTL doesn't depend on any technology, so Soft macros are also not specific to any manufacturing process.
 - These Macros are can edit before going to later stages, so that means soft macros are more flexible than hard or Firm Macros
 - Soft Macros is a Plan Group, isolated from logic hierarchy, which belongs to a certain physical hierarchy but lacks certain shape
 - Soft Macros will allow better Global Optimization
 - Final assembly may solve shape, pin, and global timing problems

Cells & Layer : IP

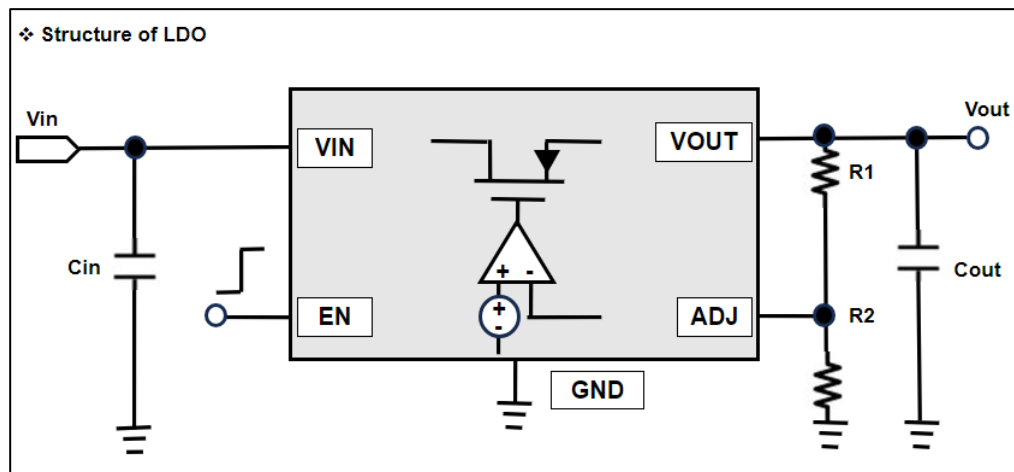
- What is IP
 - IP stands for intellectual property
 - In electronic design, IP core or block is a reusable unit of logic, cell or integrated circuit layout design that is the intellectual property on the party
 - IP cores can be licensed to another party or owned and used by a single party
 - IP cores are standalone modules that can be used in any field programmable gate array (FPGA)
- Types of IP
 - IP cores are generally licensed as either **Soft IP cores** or **Hard IP cores**
- Soft IP
 - Soft IP cores are IP blocks offered as synthesizable RTL models
 - Soft IPs are developed in one of the Hardware description language like System Verilog or VHDL
- Hard IP
 - Hard IP cores are offered as layout designs in a layout format like GDS which is mapped to a process technology and can be directly dropped by a consumer to the final layout of the chip
 - When Layout in Back End, we use Hard IP designed in GDS format
 - Hard IP cores cannot be customized for different process technologies

Cells & Layer : Types of IP

- LDO

- LDO stands for Low-Dropout-regulator
- LDO regulator is a linear regulator that can operate at a very low potential difference between the input and output voltage
- LDO is a type of power supply IC that can output a steady voltage from an input voltage and is used in a variety of electronic devices
- Advantages of an LDO over other DC-to-DC voltage regulators include the absence of switching noise, smaller device size, and greater design simplicity
- LDO usually consists of a reference, an Amplifier, and a pass element
- The disadvantage of LDO is a LDO must dissipate power, and thus heat, across the regulation device in order to regulate the output voltage

- Example of LDO

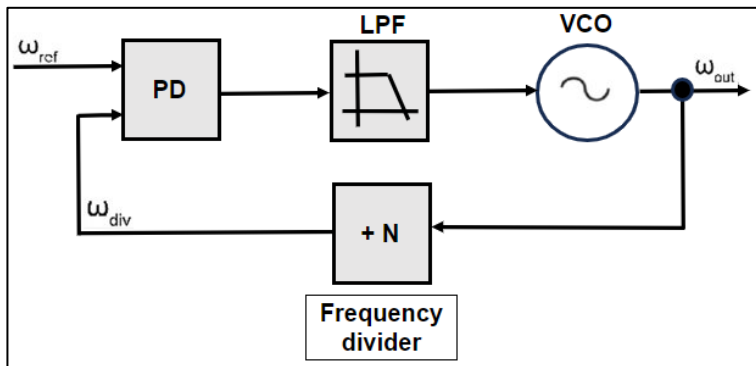


Cells & Layer : Types of IP

- PLL

- PLL is a control system that generates an output signal whose phase is related to the phase of an input signal
- The simplest type of PLL is an electronic circuit consisting of a variable frequency oscillator and a phase detector in a feedback loop
- Oscillator's frequency and phase are controlled proportionally by an applied voltage, hence the term voltage-controlled oscillator (VCO)
- The oscillator generates a periodic signal of a specific frequency, and the phase detector compares the phase of that signal with the phase of the input periodic signal, to adjust the oscillator to keep the phases matched
- PLL can be used to demodulate a signal, recover a signal from a noisy communication channel, generate a stable frequency at multiples of an input frequency, or distribute precisely timed clock pulses in digital logic circuits such as microprocessors
- Single integrated circuit can provide a complete phase-locked-loop building block, the technique is widely used in modern electronic devices
- PLL have output frequencies from a fraction of a Hz up to many GHz

- Example of PLL

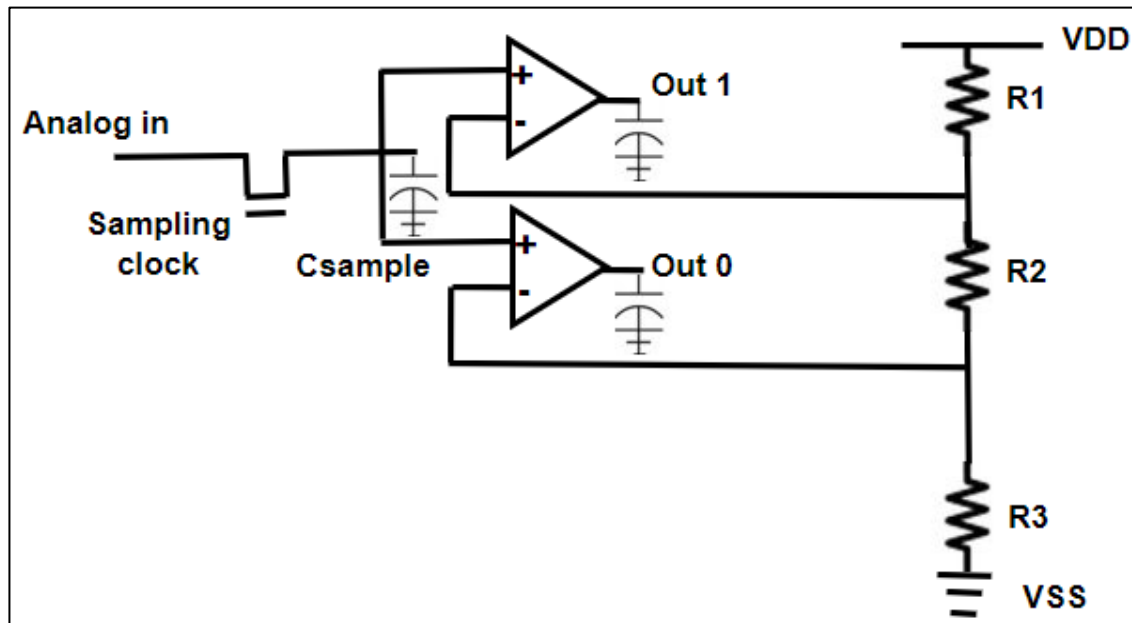


Cells & Layer : Types of IP

- ADC converter

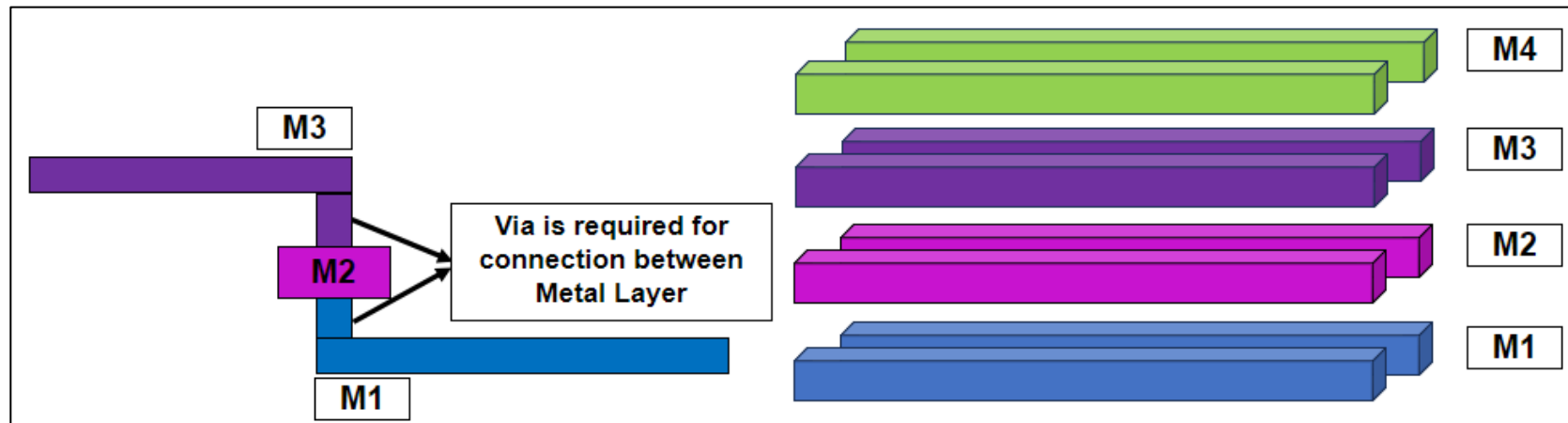
- ADC stands for Analog to Digital Converter
- ADC is an electronic device used for converting an analog signal into a digital signal
- In the real world, every real quantity such as (voice, temperature, weight) exists in the analog state. And it cannot be processed by any digital device such as a computer. These analog quantities are converted into digital form so that a digital device can process it. This conversion is done using analog to digital converter (ADC)

- Example of ADC converter



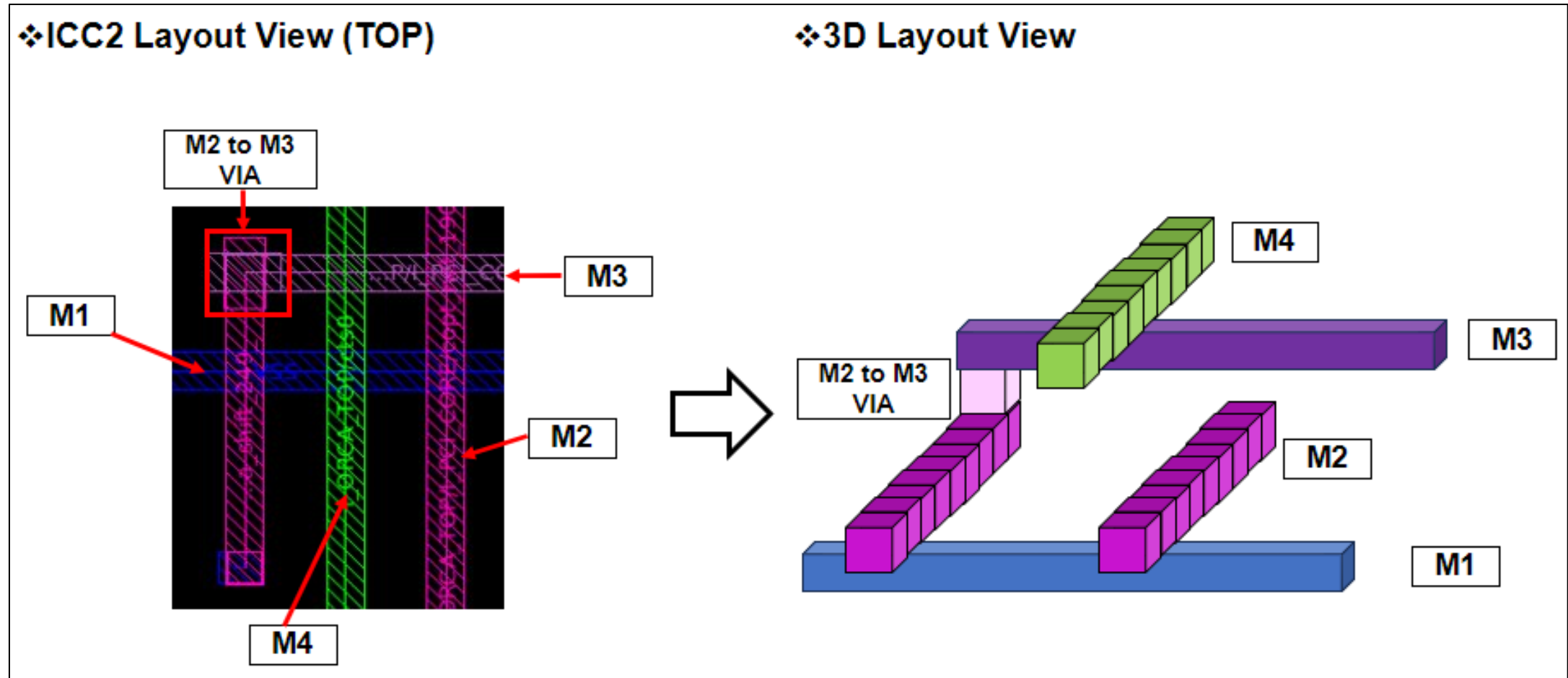
Cells & Layer : Metal Layers

- What is Metal Layers
 - Metal Layers are used to route any PG/Clock/Signal nets
 - Metal Layers connect the points of the two ends
 - There can be many numbers of metal layers which has been used to complete the routing
 - The number of metal layers to be used depend upon the foundry and technology node
 - The metal layers are drawn in such a way that from M1-M14 we will have Horizontal and vertical metal layers
 - That means M1 is Horizontal, M2 is vertical, M3 is Horizontal, M4 is vertical and so on (M1 is generally used standard cell rail)
 - To connect with Metal Layers, we need VIAs which connects two metal layers
- Example of Metal Layers



Cells & Layer : Metal Layers

- Example of Metal Layers Connections



Cells & Layer : VIAs

- What is VIAs
 - VIAs are used to connect between different metal layers
 - To connect between different metal layers, we need poly layer along with the metal layers that we are going to connect
 - A via forms a connection between overlapping geometries on different layers through a cut layer, and is formed by geometries on all three layers
- 3 Types of VIAs
 - Single VIA
 - Array VIA
 - Stacked VIA

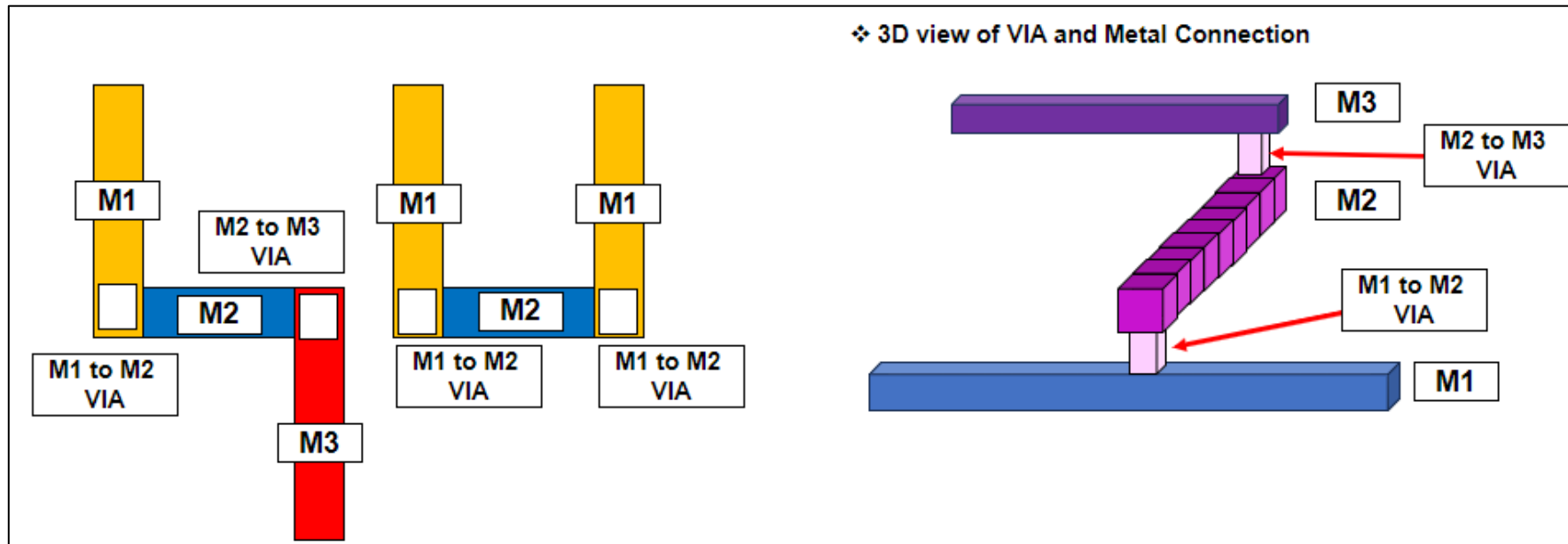
Cells & Layer : types of VIAs

- Single VIA

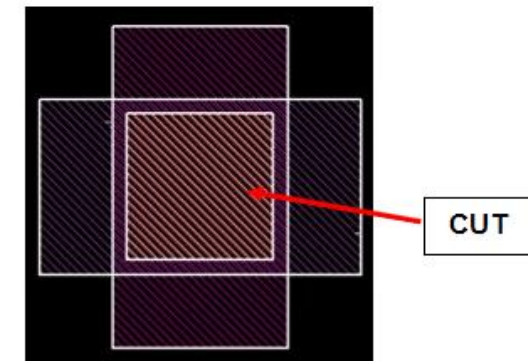
- What is Single VIA

Single via is a type of via where we have only one poly connection between the two connecting metal layers

- Example of Single VIA



❖ Array VIA



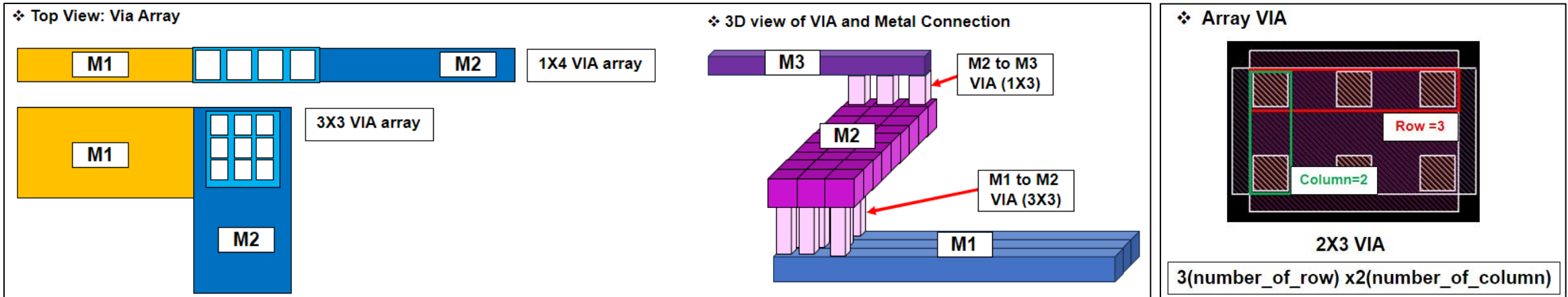
1X1 VIA

1(number_of_row) x 1(number_of_column)

Cells & Layer : types of VIAs

- Array VIA

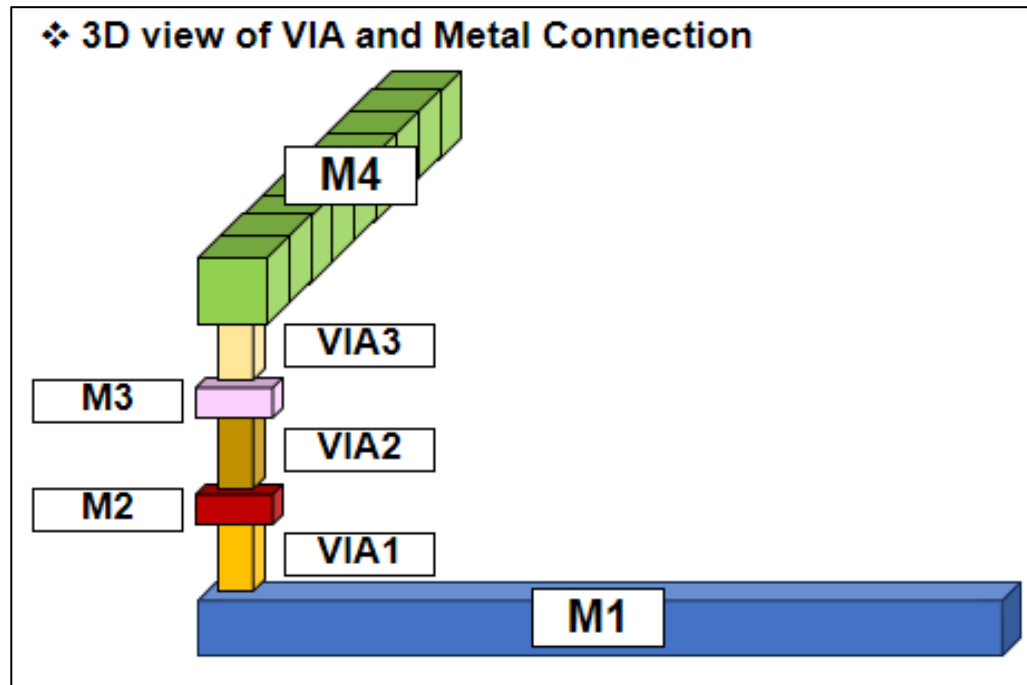
- What is Array VIA
 - Array vias are usually used when a wide wire changes layer (power or ground wire)
 - Power and Ground wires are wide because they have to carry a lot of current
 - Single via is not able to carry sufficient current so we need either a much bigger via, or more of the regular small vias connected in parallel
 - when wide wires change layer they do so through multiple vias arranged in an array pattern
-
- Example of Array VIA



Cells & Layer : types of VIAs

- Stacked VIA

- What is Stacked VIA
 - Stacked VIAs are a type of via used to connect circuits between different layers across 3+ circuit layers
 - Stacked vias are used to provide support for the thicker copper metal lines running above in bond pad structures
- Example of Stacked VIA



WEEK 1 : The Basic overview of ICC2 Tool LAB

IC Compiler GUI : Before Setting Invoke ICC2

- Edit your ICC2 path in setup.csh

```
< user@linux 00_create_ndm >> gvim setup.csh
```

```
# SYNOPSYS IC Compiler II: Block-level Implementation Workshop environment - model setup file
#
# The SYNOPSYS environment points to where the tool is installed
setenv ICC2_HOME /tools/synopsys/icc2/2021.06-SP5 ;# edit your icc2 path directory

# This variable needs to point to the license server
setenv SNPSLMD_LICENSE_FILE 27000@license_server

# Adjust exec path
set path = ( . $path $ICC2_HOME/bin )
```

- Source setup.csh in your run directory

```
< user@linux 00_create_ndm >> source setup.csh
```

- ❖ To use ICC2, you must source the setup.csh file in the executable directory of each LAB, and you must edit the ICC2 path of the file

IC Compiler GUI : Launch IC compiler II

1. Invoke tool (icc2 in your work directory)

- **Recommend icc2 version is the icc2/2021.06-SP5**

```
>> source ./setup.csh      ;# set icc2 environment and search path / edit ICC2_HOME variable for install directory of icc2
```

```
>> icc2_shell              ;# invoking icc2 tool
```

```
<kjinsoo@us01odcvde31334 00_create_ndm >> icc2_shell

                                IC Compiler II (TM)

                                Version 0-2018.06-SP5 for linux64 - Jan 22, 2019
                                    -SLE

                                Copyright (c) 1988 - 2019 Synopsys, Inc.
                                This software and the associated documentation are proprietary to Synopsys,
                                Inc. This software may only be used in accordance with the terms and conditions
                                of a written license agreement with Synopsys, Inc. All other use, reproduction,
                                or distribution of this software is strictly prohibited.

                                Loading user preference file /u/kjinsoo/.synopsys icc2 gui/preferences.tcl
```

2. Start the GUI

```
icc2_shell> _start_gui
```

IC Compiler GUI : Launch IC compiler II

ICC2 GUI Window

IC Compiler II - BlockWindow.1 - Design Planning

File Task Edit Create View Select Highlight Schematic Window Help

Input mode ☐ Rectangle ☒ Rectangle Intersect ☐ Partial Selection ☐ Smart ☐ Line ☐ Enable ☐ Yes Replace Clear Query ☒ Map ☒ Annotations

IC Compiler II

No block is loaded

ICC2 Menu

- Open an existing block
- Create a new block
- Execute script

View Settings

Layout.1

Apply

100% Level 0

Objects Layers Settings

☐ Only select highlighted

Filter objects by type name

Arrange [Default]

- Block
- Core Area
- Port
- Terminal
- Cell
- Fill Cell
- Pin
- Site Array
- Site Row
- Bound
- Placement Attrac...
- Voltage Area
- Route

Console

Log History

icc2_shell>

Script Editor

Click objects or drag a box to select (Hold Ctrl to add, Shift to remove)

Terminal

Mate Terminal

```
<kjinsoo@us01odcvde31334 rundir >> cd 00_create_ndm/  
<kjinsoo@us01odcvde31334 00_create_ndm >> ls  
check_workspace.ems  icc2_output.txt  ndm_OLD  
FP_data              lm_command.log  nlib  
icc2_command.log      lm_log          OVERLAP.lef  
icc2_common_env       lm_output.txt   PreFrameCheck  
icc2_lm_command.log    lm_shell.log    script  
icc2_lm_output.txt     ndm             user_script  
<kjinsoo@us01odcvde31334 00_create_ndm >> icc2_shell -gui
```

IC Compiler II (TM)

Version S-2021.06-SP5 for linux64 - Jan 14, 2022 -SLE

Copyright (c) 1988 - 2022 Synopsys, Inc.

This software and the associated documentation are proprietary to Synopsys, Inc. This software may only be used in accordance with the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, or distribution of this software is strictly prohibited. Licensed Products communicate with Synopsys servers for the purpose of providing software updates, detecting software piracy and verifying that customers are using Licensed Products in conformity with the applicable License Key for such Licensed Products. Synopsys will use information gathered in connection with this process to deliver software updates and pursue software pirates and infringers.

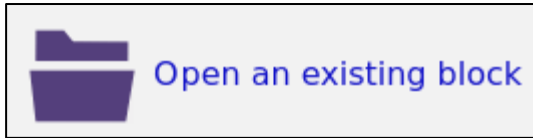
Inclusivity & Diversity - Visit SolvNetPlus to read the "Synopsys Statement on Inclusivity and Diversity" (Refer to article 000036315 at <https://solvnetplus.synopsys.com>)

Loading user preference file /u/kjinsoo/.synopsys_icc2_gui/preferences.tcl

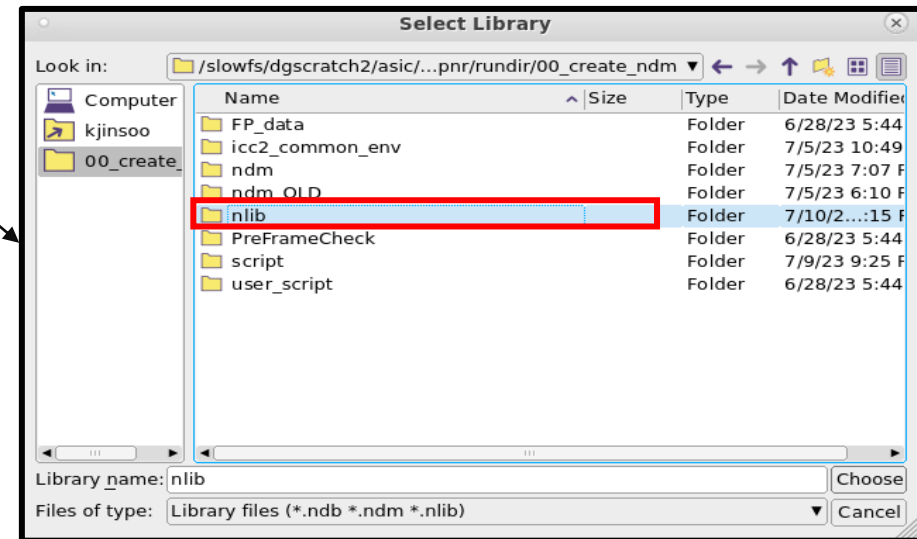
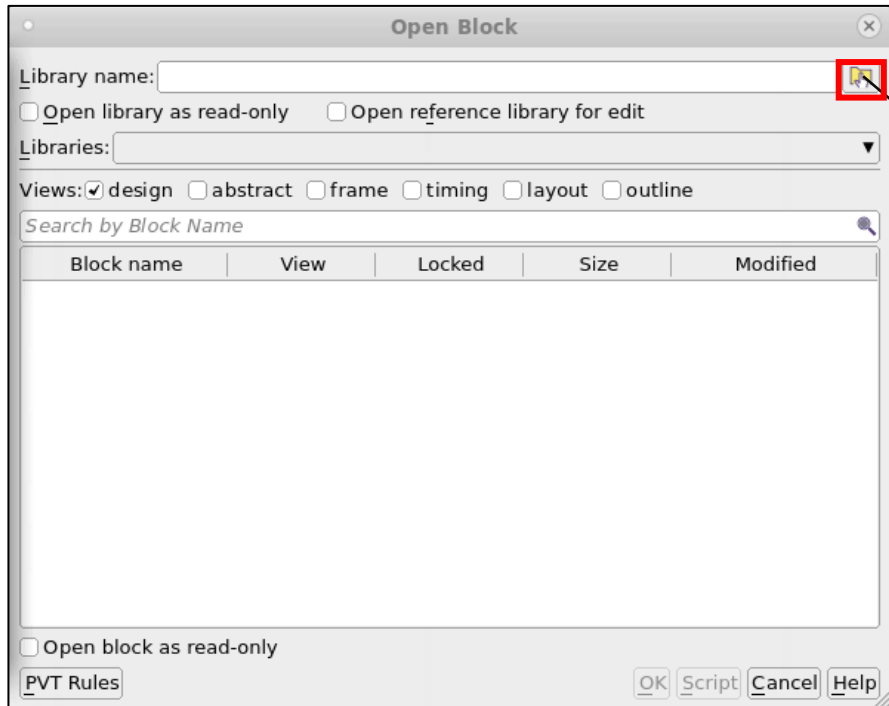
icc2_shell>

IC Compiler GUI : Open Block

1. The ICC2 BlockWindow opens and click on the symbol to open an existing block

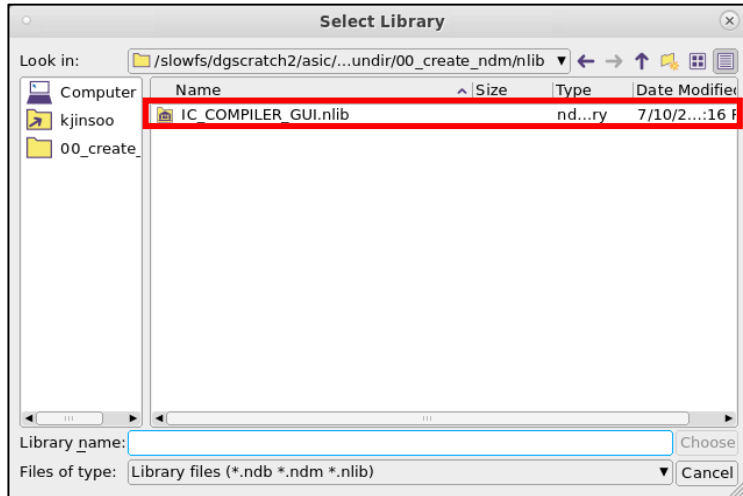


2. In the dialog that opens, click on the yellow symbol in the top right corner to choose nlib folder



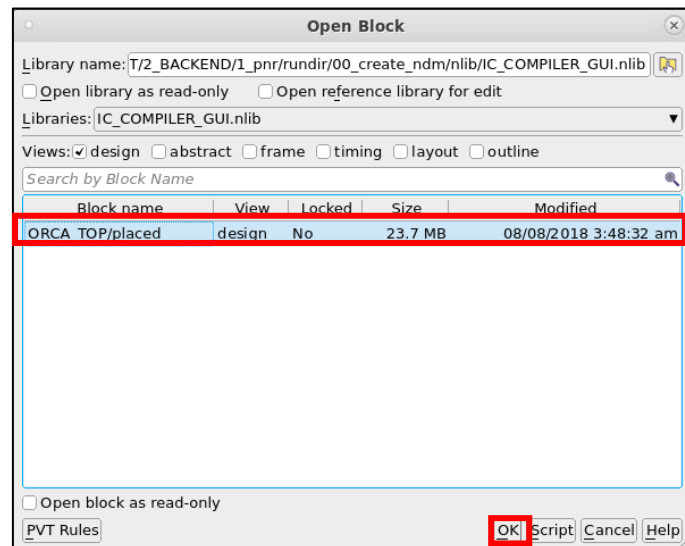
IC Compiler GUI : Open Block

3. In the dialog that opens, choose a design library called "IC_COMPILER_GUI.nlib"



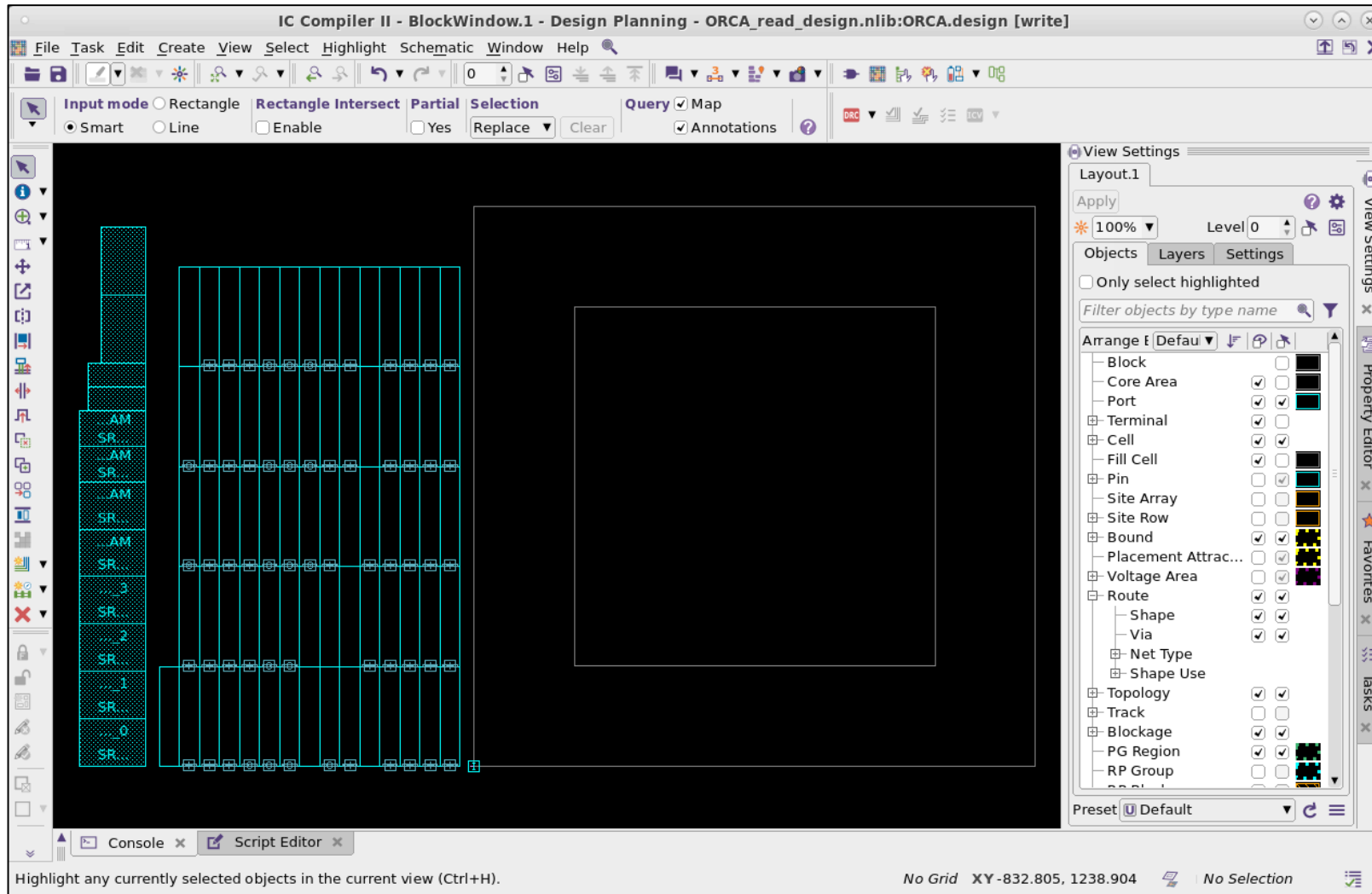
❖ Design libraries are marked with this symbol 

4. Now that a library has been chosen, select "ORCA_TOP/placed" from the list at the bottom of the Open Block dialog. Click OK. You will be presented with the layout view of the design



IC Compiler GUI : Open Block

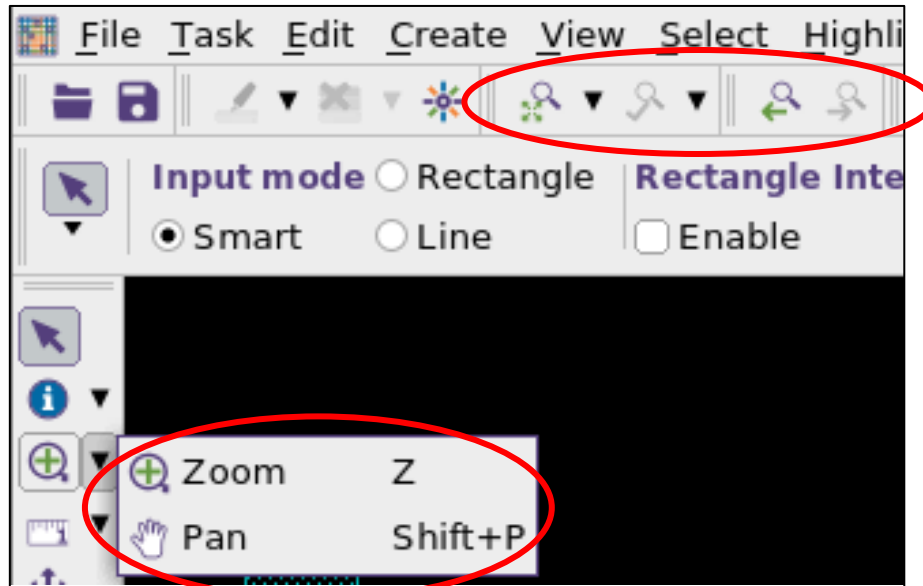
- Enlarge or maximize the GUI window. You will see that the layout view adjusts to fit the larger window



IC Compiler GUI : Navigation the Layout view

- Spend a few minutes to become familiar with the zoom and pan buttons

Hint : A short, descriptive ToolTip will pop up when a mouse pointer is held motionless over a button



- To exit the zoom and pan mode press the [ESC] key or pick the Selection Tool(the white arrow icon).The cursor returns to an arrow or pointer shape
- Use hot keys: Lower_case [F] or [Ctrl F] both correspond to zoom fit all(or full view), for example ([+] or [=] is zoom in 2x, [-] is zoom-out 2x)
- Zoom in on an area of interest: Lower-case [Z] and [ESC]

IC Compiler GUI : Controlling object and Layer Visibility

- You can control What types of objects are visible and/or selectable through the View Setting panel. In the following steps you will turn on visibility to some key objects one at a time, to clearly see what they represent

The screenshot shows the 'View Settings' panel in the IC Compiler GUI. The panel has tabs for 'Objects', 'Layers', and 'Settings'. The 'Objects' tab is active, showing a list of object types with checkboxes for selection and visibility. Annotations with arrows point to specific controls:

- More information:** Points to the 'Filter objects by type name' search bar.
- Selection control:** Points to the 'Selection' column of checkboxes.
- Visibility control:** Points to the 'Visibility' column of checkboxes.
- In the visibility column uncheck Route:** Points to the 'Route' row in the visibility column.
- Make pins un-selectable by unchecking the box in the selection column. Draw the same box again to see that only the macros are selected:** Points to the 'Pin' row in the selection column.
- Select Default from the pull down list to restore the default settings, Fit the view to the window [F]:** Points to the 'Preset' dropdown menu at the bottom.
- Make sure that under the Options button, Auto apply is checked: This way changes are applied immediately without having to confirm with the Apply button each time:** Points to the 'Auto apply' checkbox in the 'Options' section.

The 'View Settings' panel includes an 'Apply' button, a zoom level of 100%, and a level indicator. The object list includes: Block, Core Area, Port, Terminal, Cell, Fill Cell, Pin, Site Array, Site Row, Bound, Placement Attrac..., Voltage Area, Route, Shape, Via, Net Type, Shape Use, Topology, Track, Blockage, PG Region, and RP Group.

IC Compiler GUI : Controlling object and Layer Visibility

View Settings
Layout.1
Apply
100%
Level 0

Objects Layers Settings

☒ Datatypes
Filter layers by name/number

Layers
Arrange By: Mask

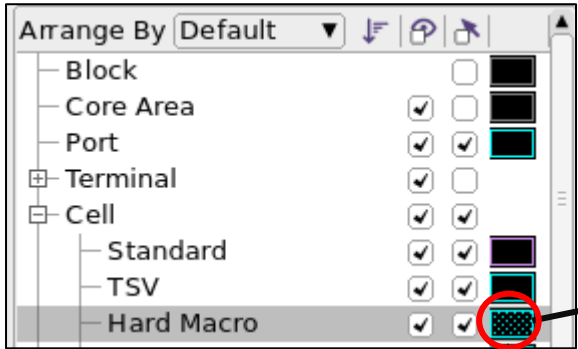
Layer Name	Visible	Selectable	Mask
NWELL (1)	☒	☒	[Mask Icon]
PO (10)	☒	☒	[Mask Icon]
PODMY (10:1)	☒	☒	[Mask Icon]
CO (28)	☒	☒	[Mask Icon]
M1 (11)	☒	☒	[Mask Icon]
M1DMY (11:1)	☒	☒	[Mask Icon]
VIA1 (12)	☒	☒	[Mask Icon]
M2 (13)	☒	☒	[Mask Icon]
M2DMY (13:1)	☒	☒	[Mask Icon]
VIA2 (14)	☒	☒	[Mask Icon]
M3 (15)	☒	☒	[Mask Icon]
M3DMY (15:1)	☒	☒	[Mask Icon]
VIA3 (16)	☒	☒	[Mask Icon]
M4 (17)	☒	☒	[Mask Icon]
M4DMY (17:1)	☒	☒	[Mask Icon]
VIA4 (18)	☒	☒	[Mask Icon]
M5 (19)	☒	☒	[Mask Icon]
M5DMY (19:1)	☒	☒	[Mask Icon]

Preset **Default**

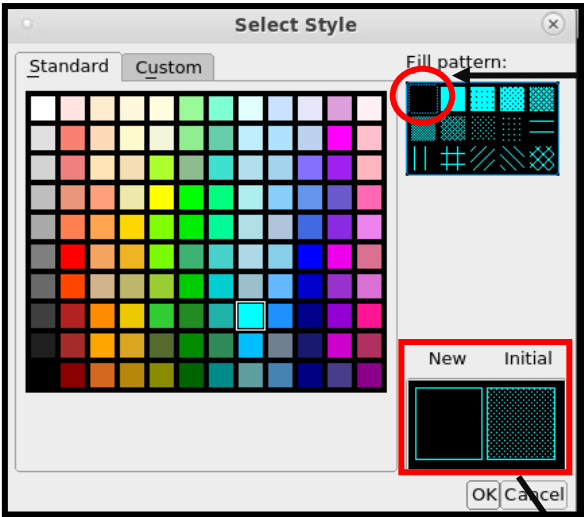
Annotations:

- Select the Layers tab, which can be used to fine tune the routing visibility further on a layer-by-layer basis
- To see only the actual routing layers, click on the filter button and uncheck "All Layer Types" first to clear everything, then check "Routing"
- Just as with the Objects, you can control visibility as well as selectability of the routing layers by using the checkmarks
- In addition, you can click on the individual items for finer control. For example, click on the up/down arrow next to M2 to toggle track visibility. In addition, you can control the visibility of Shape, Via, Terminal etc...
- Notice that there are more columns which you can control. Use the horizontal scroll bar to see them all

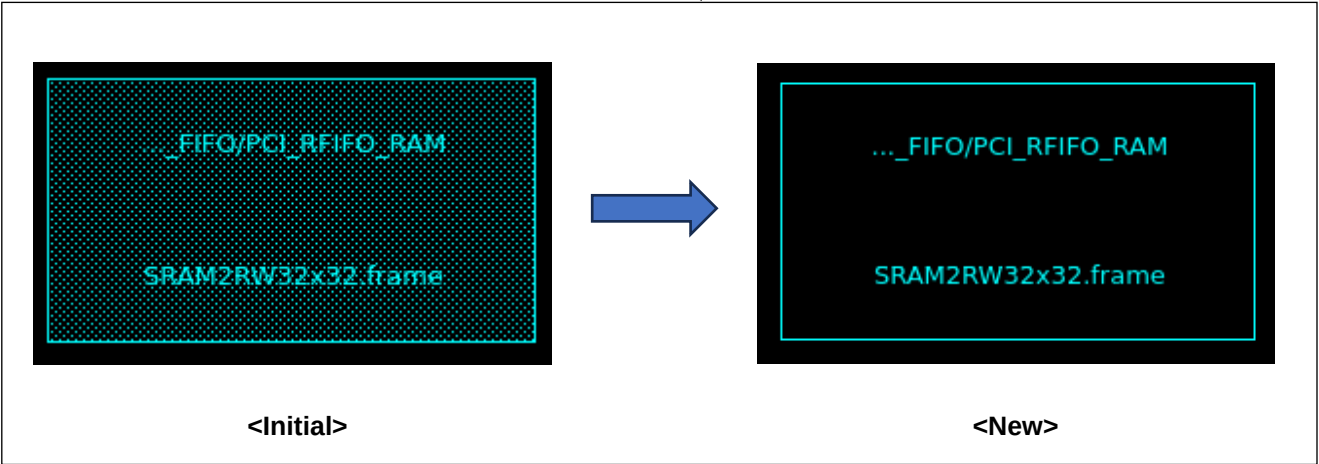
IC Compiler GUI : Controlling object and Layer Visibility



To better see the objects inside the macros,
remove the fill pattern of the macro shapes:
In the Objects tab of the View Settings panel,
expand Cell, then click on the aquamarine outline
pattern of Hard Macro.

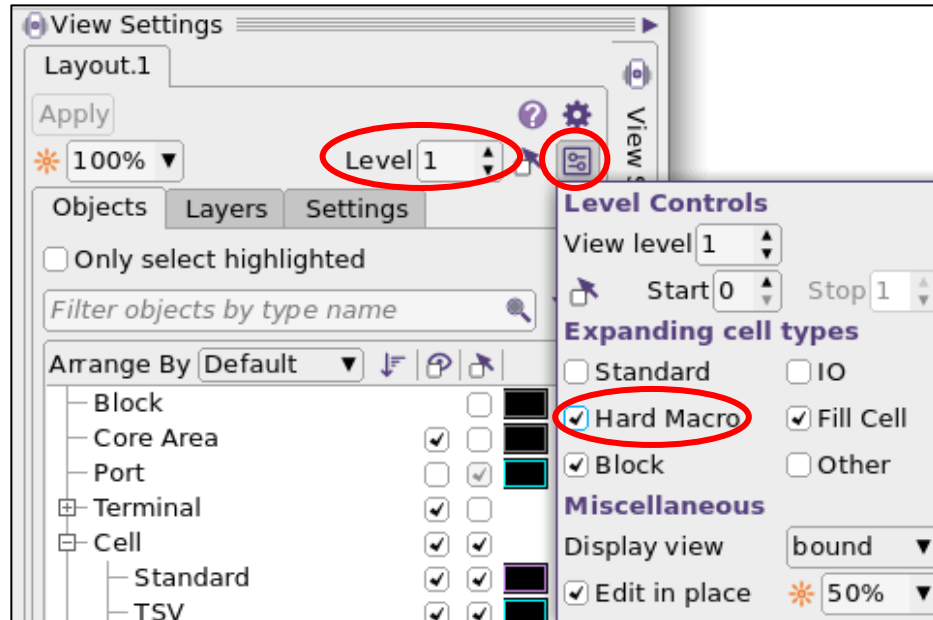


In the Select Style dialog, go to the Fill pattern section in the upper-right, and select the black(no fill) pattern in the upper-left, then click OK



IC Compiler GUI : Controlling object and Layer Visibility

- Increase the viewing Level from 0 to 1, then click on the Hierarchy Settings button to the right of the Level field, and check Hard Macro



<Level 0>

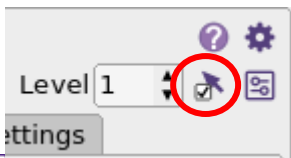


<Level 1>



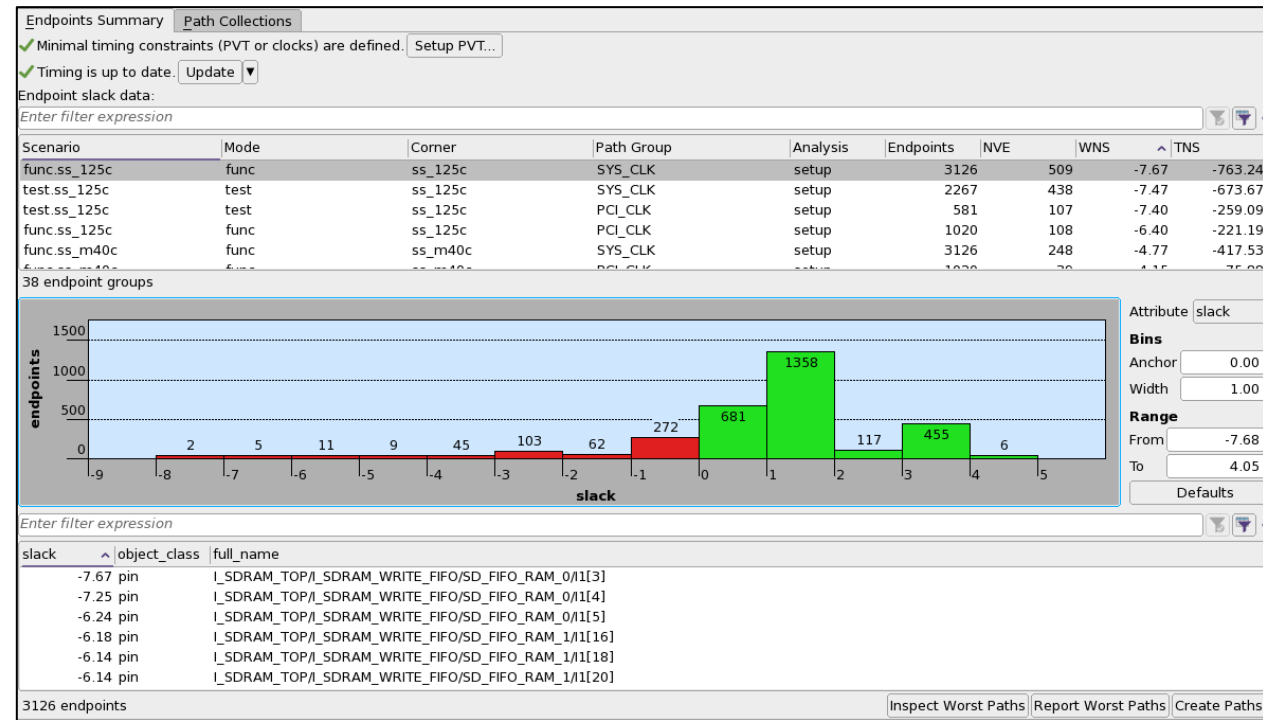
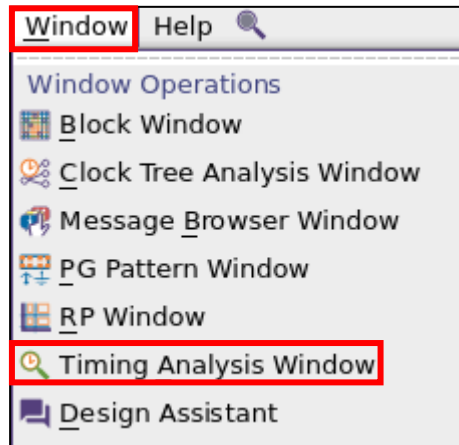
<multiple levels>

- If you zoom into one of the RAMs you should see the structures inside the macro. You can turn layer visibilities off/on to see individual layers. Since these are frame views, what you are seeing are mostly routing blockages
- You can also interact with structures within cells: Click on the Multiple Levels Active button shown here. Now you can hover over objects within cells, and analyze them. This functionality is used extensively in hierarchical design planning, where you can perform physical manipulation at multiple physical design levels at the same time



IC Compiler GUI : Timing Analysis

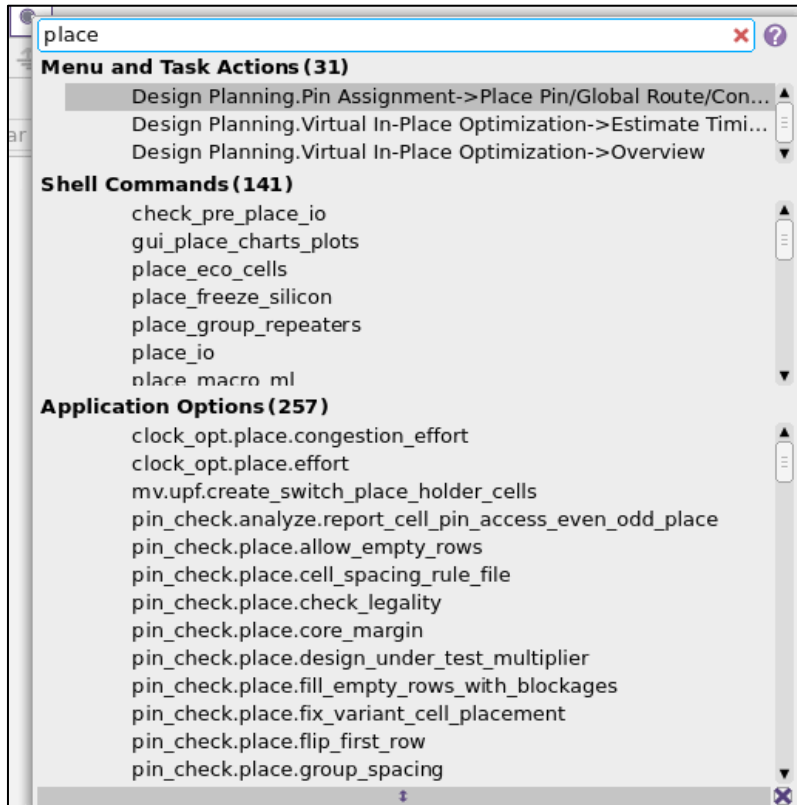
- Have a look at the timing of this design. Use the menu Window – Timing Analysis Window. In the new window that appears, press the Update button in the top left to update the timing



- Once timing is up-to date, you will see a histogram for the timing distribution. Violating paths show up as bars in red on the far left and positive slack paths in green. If you want to see details, you can click on the bars to see the timing paths involved

IC Compiler GUI : Getting Help

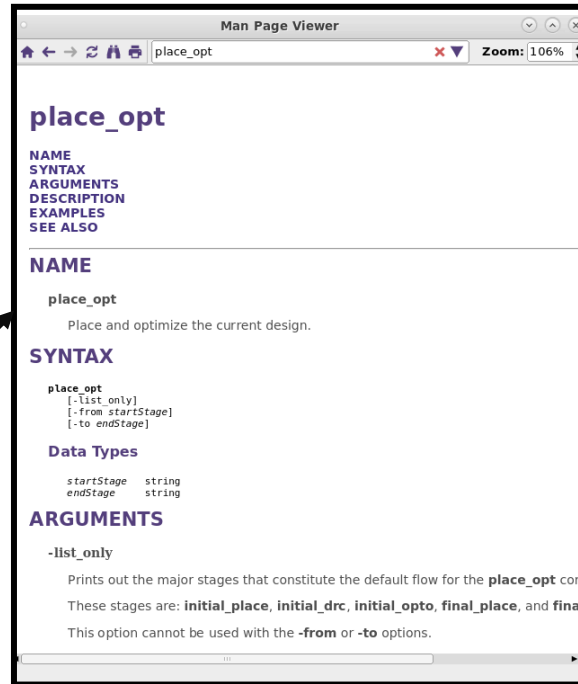
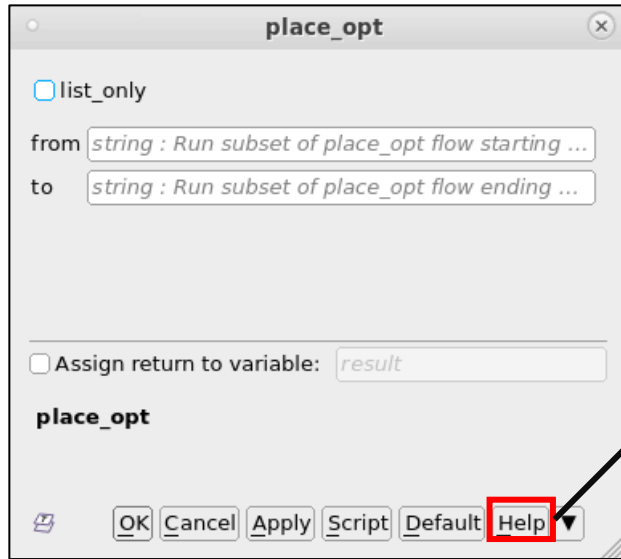
- The quickest way to get help is to use command search
- Access command search by using the keyboard shortcut lower-case [H] or by clicking on the magnifying glass  at the top of the block window, on the right end of the menus
- In the window that appears, begin typing "place". Notice that as you type, ICC2 lists all the menus, commands and application options that relate to that search string



IC Compiler GUI : Getting Help

- Select one of the Shell Commands, for example, (A second window will open, titled "place_opt")

place_opt



- In this window you can change the settings of shell command options, or you can search for them (Explore this window a little to become familiar with it)
- You can also open the manual page by click the Help button, which allows you to view detailed descriptions of Shell Command

IC Compiler GUI : Getting Help

- To list the available options for place_opt (note that you can also use tab completion on the place* command after help):

```
help place_opt -verbose
or
help place_opt -v
or
place_opt -help
```

```
icc2_shell> help place_opt -verbose
place_opt          # place-opt Optimization
[-list_only]      (List sub-steps of place_opt flow [ initial_place initial_drc
initial_opto final_place final_opto ])
[-from ]          (Run subset of place_opt flow starting with specified step)
[-to ]            (Run subset of place_opt flow ending with specified step)
```

- To get a full help manual page – a detailed description of the command and all of its options, type;

```
man place_opt
```

```
icc2_shell> man place_opt
2. Synopsys Commands
```

	place_opt	Command Reference
--	-----------	-------------------

```
NAME
    place_opt
    Place and optimize the current design.

SYNTAX
    place_opt
        [-list_only]
        [-from startStage]
        [-to endStage]

Data Types
    startStage  string
    endStage    string

ARGUMENTS
    -list_only
        Prints out the major stages that constitute the default flow for
        the place_opt command.

        These stages are: initial_place, initial_drc, initial_opto,
        final_place, and final_opto.

        This option cannot be used with the -from or -to options.
```

IC Compiler GUI : Getting Help

- Now let's say you need help on a specific application option. But again, you don't remember its exact name, but it pertains to Place. To list all application options that start with place, enter:

```
report_app_options place*
```

```
icc2_shell> report_app_options place*
*****
Report : app_option
Design : NO_DESIGN
Version: S-2021.06-SP5
Date   : Tue Jul 11 18:02:37 2023
*****
Name                                     Type
Value      User-value  User-default System-default Scope  Status
-----
-----
place.coarse.advanced_node_congestion_driven_max_util  bool
--              --              --              false      block  normal
place.coarse.auto_density_control                      enum
--              --              --              true       block  normal
place.coarse.auto_timing_control                       bool
--              --              --              true       block  normal
```

- You can also get additional help for an error or warning message, using the unique message code, for example

```
man ZRT-536
```

```
icc2_shell> man ZRT-536
N. Messages                                     Command Reference

                                messages

NAME
    ZRT-536 (warn) There is no constraint to limit maximum number of secondary power and ground (PG) pins in a cluster.

DESCRIPTION
    The option value of route.common.number_of_secondary_pg_pin_connections in set_app_options is 0. The routing/drc-check behavior for secondary PG pin will be exactly the same as signal pin.

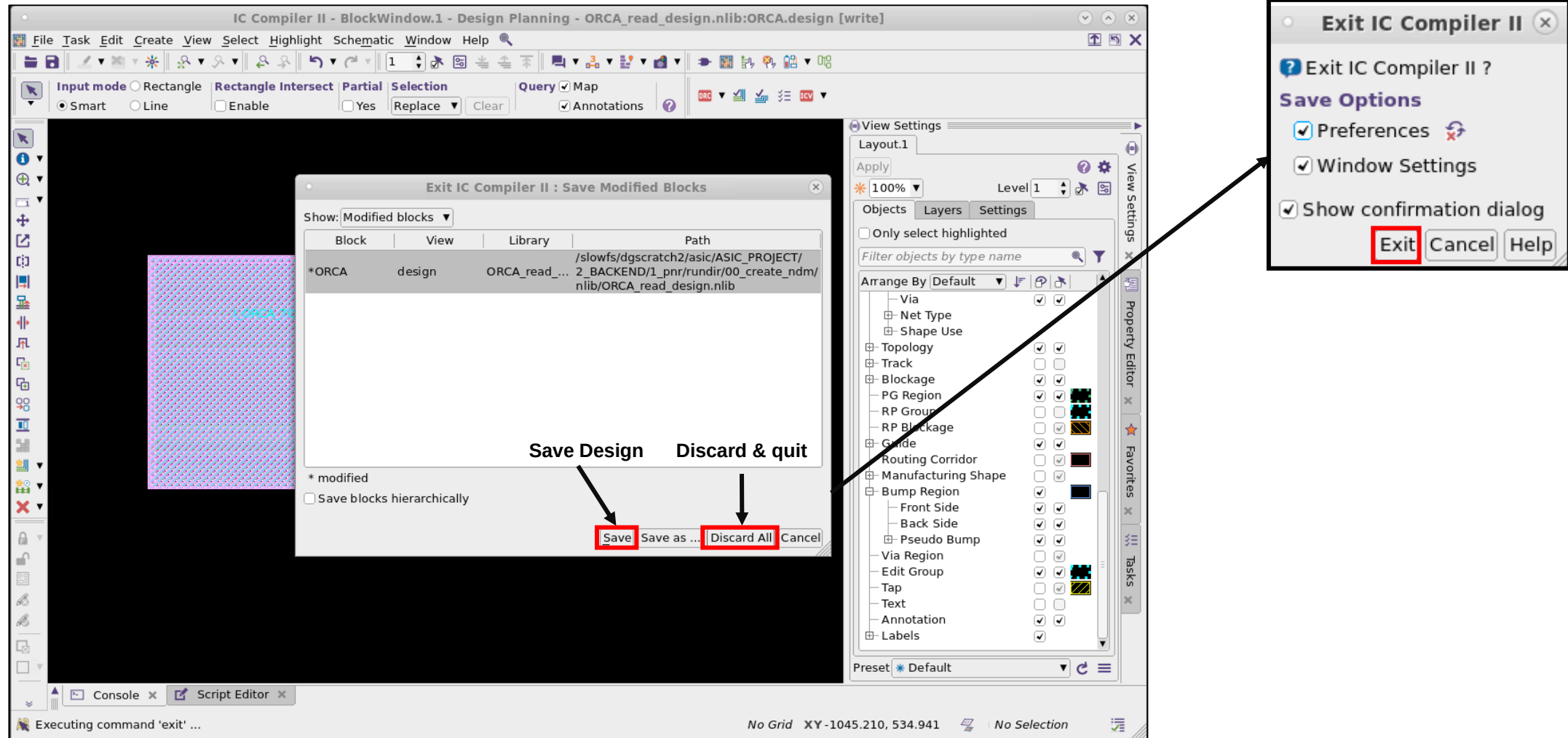
WHAT NEXT
    Please set option value of route.common.number_of_secondary_pg_pin_connections in set_app_options

                                Version S-2021.06-SP5
                                Copyright (c) 2022 Synopsys, Inc. All rights reserved.
```

IC Compiler GUI : Exit ICC2

- Exit ICC2

```
icc2 shell> exit
```



WEEK 2 : LAB for create NDM

Create NDM : Prepare library db files

- Prepare db library and lvt files for create NDM files

Set environment for your working directory : `"/slowfs/dgscratch2/asic/ASIC_PROJECT"`

```
setenv WORK_DIR "your working directory"
```

```
>> setenv WORK_DIR "/slowfs/dgscratch2/asic/ASIC_PROJECT"
```

```
>> echo $WORK_DIR
```

Change to DBS directory as below

```
>> cd $WORK_DIR/2_BACKEND/1_pnr/libdir/DBS/
```

```
>> pwd
```

Link lvt std cell library to the DBS directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_lvt/db_nldm/saed32lvt_ff1p16v125c.db ./
```

;#(./ Means current directory)

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_lvt/db_nldm/saed32lvt_ff1p16vn40c.db ./
```

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_lvt/db_nldm/saed32lvt_ss0p95v125c.db ./
```

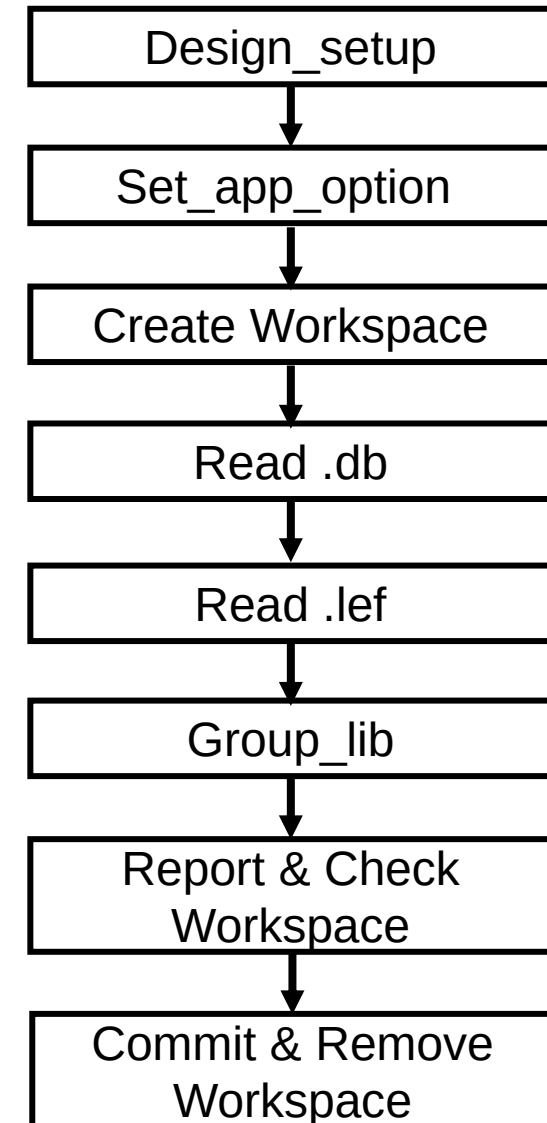
```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_lvt/db_nldm/saed32lvt_ss0p95vn40c.db ./
```

```
>> ls *lvt*.db ;# (display only lvt std library of corner case)
```

Create NDM

- Create NDM Overview

- This module provides an overview of some of the key steps in create NDM :
- **Defining**
 - Tech file (.tf)
 - .db
 - .lef
- **Overview of Create NDM (directory, input files, script)**
- **Create NDM**



Create NDM : Prepare library db files

- Prepare db library and lef files for create NDM files

Link hvt std cell library to the DBS directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_hvt/db_nldm/saed32lvt_ff1p16v125c.db ./ ;#(./ Means current directory)
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_hvt/db_nldm/saed32lvt_ff1p16vn40c.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_hvt/db_nldm/saed32lvt_ss0p95v125c.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_hvt/db_nldm/saed32lvt_ss0p95vn40c.db ./
>> ls *hvt*.db ;# (display only hvt std library of corner case)
```

Link rvt std cell library to the DBS directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32lvt_ff1p16v125c.db ./ ;#(./ Means current directory)
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32lvt_ff1p16vn40c.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32lvt_ss0p95v125c.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32lvt_ss0p95vn40c.db ./
>> ls *rvt*.db ;# (display only rvt std library of corner case)
```

Link IO std cell library to the DBS directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/io_std/db_nldm/saed32io_fc_ff1p16v125c_2p75v.db ./ ;#(./ Means current directory)
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/io_std/db_nldm/saed32io_fc_ff1p16vn40c_2p75v.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/io_std/db_nldm/saed32io_fc_ss0p95v125c_2p25v.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/io_std/db_nldm/saed32io_fc_ss0p95vn40c_2p25v.db ./
>> ls *hvt*.db ;# (display only hvt std library of corner case)
```

Create NDM : Prepare library db files

- Prepare db library and lef files for create NDM files

Link MEMORY(SRAM) cell library to the DBS directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/sram/db_nldm/saed32sram_ff1p16v125c.db ./ ;#(./ Means current directory)
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/sram/db_nldm/saed32sram_ff1p16vn40c.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/sram/db_nldm/saed32sram_ss0p95v125c.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/sram/db_nldm/saed32sram_ss0p95vn40c.db ./
>> ls *sram*.db ;# (display only sram library of corner case)
```

Link PLL cell library to the DBS directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/pll/db_nldm/saed32pll_ff1p16v125c_2p75v.db ./ ;#(./ Means current directory)
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/pll/db_nldm/saed32pll_ff1p16vn40c_2p75v.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/pll/db_nldm/saed32pll_ss0p95v125c_2p25v.db ./
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/pll/db_nldm/saed32pll_ss0p95vn40c_2p25v.db ./
>> ls *pll*.db ;# (display only pll library of corner case)
```

Display the db libraries in your DBS directory

```
< user@linux DBS >> pwd
/slowfs/dgscratch2/asic/ASIC_PROJECT/2_BACKEND/1_pnr/libdir/DBS
< user@linux DBS >> ls
saed32hvt_ff1p16v125c.db      saed32io_fc_ss0p95v125c_2p25v.db  saed32pll_ff1p16v125c_2p75v.db  saed32rvt_ss0p95v125c.db
saed32hvt_ff1p16vn40c.db      saed32io_fc_ss0p95vn40c_2p25v.db  saed32pll_ff1p16vn40c_2p75v.db  saed32rvt_ss0p95vn40c.db
saed32hvt_ss0p95v125c.db      saed32lvt_ff1p16v125c.db          saed32pll_ss0p95v125c_2p25v.db  saed32sram_ff1p16v125c.db
saed32hvt_ss0p95vn40c.db      saed32lvt_ff1p16vn40c.db          saed32pll_ss0p95vn40c_2p25v.db  saed32sram_ff1p16vn40c.db
saed32io_fc_ff1p16v125c_2p75v.db  saed32lvt_ss0p95v125c.db          saed32rvt_ff1p16v125c.db        saed32sram_ss0p95v125c.db
saed32io_fc_ff1p16vn40c_2p75v.db  saed32lvt_ss0p95vn40c.db          saed32rvt_ff1p16vn40c.db        saed32sram_ss0p95vn40c.db
```

Create NDM : Prepare lef files

- Prepare db library and lef files for create NDM files

Set environment for your working directory : “/slowfs/dgscratch2/asic/ASIC_PROJECT” (example for lab)

```
setenv WORK_DIR "your working directory"
```

```
>> setenv WORK_DIR "/slowfs/dgscratch2/asic/ASIC_PROJECT"
```

```
>> echo $WORK_DIR
```

Change to LEF directory as below

```
>> cd $WORK_DIR/2_BACKEND/1_pnr/libdir/LEF/
```

```
>> pwd
```

Link lvt std cell lef to the LEF directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_lvt/lef/saed32nm_lvt_1p9m.lef ./ ;#(./ Means current directory)
```

```
>> ls *lvt*.lef (display only lvt std lef)
```

Link hvt std cell lef to the LEF directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_hvt/lef/saed32nm_hvt_1p9m.lef ./ ;#(./ Means current directory)
```

```
>> ls *hvt*.lef (display only hvt std lef)
```

Create NDM : Prepare lef files

- Prepare db library and lef files for create NDM files

Link rvt std cell lef to the LEF directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/stdcell_rvt/lef/saed32nm_rvt_1p9m.lef ./ ;#(./ Means current directory)
>> ls *rvt*.lef (display only rvt std lef)
```

Link IO std cell lef to the LEF directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/io_std/lef/saed32_io_fc_all.lef ./ ;#(./ Means current directory)
>> ls *io*.lef (display only IO std lef)
```

Link MEMORY(SRAM) cell lef to the LEF directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/sram/lef/saed32sram.lef ./ ;#(./ Means current directory)
>> ls *sram*.lef (display only sram lef)
```

Link PLL cell lef to the LEF directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/lib/pll/lef/saed32_PLL.lef ./ ;#(./ Means current directory)
>> ls *pll*.lef (display only pll lef)
```

Display the lef libraries in your LEF direvctory

```
< user@linux LEF >> pwd
/slowfs/dgscratch2/asic/ASIC_PROJECT/2_BACKEND/1_pnr/libdir/LEF
< user@linux LEF >> ls
saed32 io fc all.lef saed32nm hvt 1p9m.lef saed32nm lvt 1p9m.lef saed32nm rvt 1p9m.lef saed32 PLL.lef saed32sram.lef
```

Create NDM : Prepare technology files

- Prepare technology file for create NDM files

Set environment for working directory : “/slowfs/dgscratch2/asic/ASIC_PROJECT”

```
setenv WORK_DIR "your working directory"
```

```
>> setenv WORK_DIR "/slowfs/dgscratch2/asic/ASIC_PROJECT"
```

```
>> echo $WORK_DIR
```

Change to TECH directory as below

```
>> cd $WORK_DIR/2_BACKEND/1_pnr/libdir/TECH/
```

```
>> pwd
```

Link technology file to the DBS directory

```
>> ln -s $WORK_DIR/DK/SAED32_EDK/tech/milkyway/saed32nm_1p9m_mw.tf ./
```

;#(./ Means current directory)

```
>> ls *saed32nm*.tf (display only technology file)
```

Display the .tf libraries in your TECH directory

```
< user@linux TECH >> pwd
/slowfs/dgscratch2/asic/ASIC_PROJECT/2_BACKEND/1_pnr/libdir/TECH
< user@linux TECH >> ls
saed32nm_1p9m_mw.tf
```

Create NDM : Review create_ndm.tcl

- Review script for create ndm

1. Set environment for working directory : "/slowfs/dgscratch2/asic/ASIC_PROJECT"

```
setenv WORK_DIR "your working directory"  
>> setenv WORK_DIR "/slowfs/dgscratch2/asic/ASIC_PROJECT"  
>> echo $WORK_DIR
```

create_ndm.tcl path is as below

```
>> ls $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/00_create_ndm.tcl ;#(full script of LAB)  
>> ls $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/01_source_design_setup.tcl ~ 08_commit_and_remove_workspace.tcl ;#(LAB script)
```

2. Change to " \$WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script " directory

```
>> cd $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script
```

3. Review the "create_ndm.tcl number 01~08" with gvim editor (If you want to see a full script, open 00_create_ndm.tcl)

```
< user@linux script >> gvim 00_create_ndm.tcl
```

My Working directory

Open file
with gvim

File name

Create NDM : Introduction to create_ndm.tcl

- Review and check the variables in each scripts
- Go to the Script Directory**

```
<kjinsoo@us01odcvde31334 script >> pwd
/slowfs/dgscratch2/asic/ASIC_PROJECT/2_BACKEND/1_pnr/rundir/00_create_ndm/script
<kjinsoo@us01odcvde31334 script >> ls
00_create_ndm.tcl      03_create_workspace.tcl  06_group_libs.tcl
01_source_design_setup.tcl  04_read_db.tcl          07_report_and_check_workspace.tcl
02_source_set_app_option.tcl 05_read_lef.tcl         08_commit_and_remove_workspace.tcl
```

- Review 01_source_design_setup.tcl**

```
#set step

set step "ndm_preperation"

#source design_setup.tcl

source -echo -v ./icc2_common_env/design_setup.tcl
```

- Review 02_source_set_app_option.tcl**

```
#Ignore only waive errors when this step

set sh_continue_on_error true

#source script with a collection of options to create ndm

source -echo -v ./icc2_common_env/ndm_preperation.tcl
```

ERROR (LEFPARS-1508): TYPE statement is a required statement in a LAYER and it is not defined.
Error in file /slowfs/dgscratch2/asic/ASIC_PROJECT/2_BACKEND/1_pnr/rundir/00_create_ndm/OVERLAP
.lef at line 2, on token OVERLAP.
Error: read lef file '/slowfs/dgscratch2/asic/ASIC_PROJECT/2_BACKEND/1_pnr/rundir/00_create_ndm
/OVERLAP.lef' failed. (LEFR-002)

Create NDM : Introduction to create_ndm.tcl

- Review and check the variables in each scripts
 - **Review 03_create_workspace.tcl**

```
#Set tech, lib_lef, lib_db variables

set lib_tech "$data(tech)"
set lib_lef "$data(lefs)"
set lib_db "$data(dbs)"

#Create_workspace

create_workspace -flow exploration \
                 -technology $lib_tech EXPLORE -scale_factor 1000

echo "create_workspace was done at [date]"
```

- **Review 04_read_db.tcl**

```
#Read library db files

if {$lib_db != " "} {
    read_db $lib_db
}

set out_file "./OVERLAP.lef"
set fout [open $out_file "w"]
puts $fout "LAYER OVERLAP"
puts $fout "END OVERLAP"
close $fout
```

Create NDM : Introduction to create_ndm.tcl

- Review and check the variables in each scripts

- **Review 05_read_lef.tcl**

```
#Read library lef files

set out_file "./OVERLAP.lef"
set fout [open $out_file "w"]
puts $fout "LAYER OVERLAP"
puts $fout "END OVERLAP"
close $fout

set overlap_lef $out_file
if {$lib_lef != " "} {
    read_lef $overlap_lef
    read_lef $lib_lef
}
```

- **Review 06_group_libs.tcl**

```
#Group libs

group_libs

#Writes library preperation script files for the specified library workspaces

write_workspace -file ${NDM_DIR}/exploration.tcl
```

Create NDM : Introduction to create_ndm.tcl

- Review and check the variables in each scripts
 - **07_report_and_check_workspace.tcl**

```
#Report your workspace and check workspace  
  
#Report  
report_workspace  
  
#Check  
check_workspace
```

- **08_commit_and_remove_workspace.tcl**

```
#Finishes the process of creating a library and removing the workspace from memory  
  
#commit  
commit_workspace  
  
#remove  
remove_workspace  
  
#move ndm files to the ndm directory  
  
sh mv *.ndm ${NDM_DIR}  
  
#exit icc2_lm_shell  
  
exit
```

Create NDM : Open design_setup.tcl & ndm_preperation.tcl

- Review script for create ndm setup

1. Set environment for working directory : "/slowfs/dgscratch2/asic/ASIC_PROJECT"

```
setenv WORK_DIR "your working directory"  
>> setenv WORK_DIR "/slowfs/dgscratch2/asic/ASIC_PROJECT"  
>> echo $WORK_DIR
```

In 00_create_ndm directory have following setup script

design_setup.tcl path is as below

```
>> ls $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/icc2_common_env/design_setup.tcl
```

ndm_preperation.tcl path is as below

```
>> ls $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/icc2_common_env/ndm_preperation.tcl
```

2. Change to " \$WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/icc2_common_env" directory

```
>> cd $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/icc2_common_env
```

3. Review the "design_setup.tcl & ndm_preperation.tcl" with gvim editor

```
< user@linux icc2_common_env >> gvim design_setup.tcl  
< user@linux icc2_common_env >> gvim ndm_preperation.tcl
```

Create NDM : introduction to user_design.tcl

- Review and check the variables in the user_design.tcl script

```
set WORK_DIR "/slowfs/dgscratch2/asic/ASIC_PROJECT"

set DESIGN_NAME "ORCA" ;# Name of the design to be worked on
set DESIGN_LIBRARY "${DESIGN_NAME}.nlib" ;# Name of the design library, default is ${DESIGN_NAME}.nlib

set libdir "${WORK_DIR}/2_BACKEND/1_pnr/libdir"

set data(tech) ${libdir}/TECH/saed32nm_1p9m_mw.tf ;# A technology file; TECH_FILE and TECH_LIB are mutually exclusive ways to specify technology information;
set data(tech_tlupwc) "${libdir}/TECH/"
set data(tech_tlupbc) "${libdir}/TECH/"

#std_cell lef,lib,dbs

set data(dbs)      [glob -nocomplain ${libdir}/DBS/*.db ]
set data(libs)     [glob -nocomplain ${libdir}/LIB/*.lib]
set data(lefs)     [glob -nocomplain ${libdir}/LEF/*lef ]

set data(em_constraint) "${libdir}/TECH/saed32nm_1p9m_mw.tf"

set UPF_FILE "" ;# A UPF file
set NDM_DIR "${WORK_DIR}/2_BACKEND/1_pnr/rundir/00_create_ndm/ndm"
set NDM_FILE [glob -nocomplain $NDM_DIR/*.ndm ]

set TCL_MCMC_SETUP_FILE "" ;# A Tcl script placeholder to create your corners, modes, and scenarios,

set netlist "${libdir}/NETLIST/ORCA_32.v" ;# Verilog netlist files^M

set HORIZONTAL_ROUTING_LAYER_LIST "M1 M3 M5 M7" ;# A list of horizontal routing layers; optional; should be defined if missing in tech file
set VERTICAL_ROUTING_LAYER_LIST "M2 M4 M6 M8" ;# A list of vertical routing layers; optional; should be defined if missing in tech file
set MAX_ROUTE_LAYER "M8" ;# Min routing layer name
set MIN_ROUTE_LAYER "M1" ;# Max routing layer name

set die_width "817"
set die_height "817"
set core_offset "250"
```

;
Set Lib directory path

;
Set technology file path

;
Set library db and lef files path

;
Set NDM_DIR and NDM_FILE variables

Create NDM : Introduction to ndm_preperation.tcl

- Common app options

```
set_app_options -as_user_default -name lib.physical_model.block_all -value true
set_app_options -as_user_default -name file.lef.allow_site_conflicts -value true
set_app_options -as_user_default -name file.lef.auto_rename_conflict_sites -value true
set_app_options -as_user_default -name lib.workspace.allow_commit_workspace_overwrite -value true
set_app_options -as_user_default -name lib.logic_model.allow_ccs_timing -value true
set_app_options -as_user_default -name lib.workspace.group_libs_create_slg -value false
set_app_options -as_user_default -name lib.workspace.allow_missing_related_pg_pins -value true
set_app_options -as_user_default -name lib.logic_model.auto_remove_incompatible_timing_designs -value true
set_app_options -as_user_default -name lib.workspace.remove_frame_bus_properties -value true
set_app_options -as_user_default -name lib.logic_model.require_same_opt_attrs -value false
set_app_options -as_user_default -name lib.logic_model.use_db_rail_names -value true
set_app_options -as_user_default -name lib.workspace.require_physical -value true
set_app_options -as_user_default -name lib.logic_model.auto_remove_timing_only_designs -value true
set_app_options -as_user_default -name lib.workspace.save_design_views -value false
set_app_options -as_user_default -name lib.workspace.save_layout_views -value false
set_app_options -as_user_default -name file.lef.non_real_cut_obs_mode -value true

set_app_options -name lib.workspace.group_libs_macro_grouping_strategy -value single_cell_per_lib
set_app_options -as_user_default -name lib.workspace.group_libs_naming_strategies -value "common_prefix" ;

set_app_options -list {lib.workspace.group_libs_fix_cell_shadowing false}
```

< common app option for to create ndm >

Create NDM : Use ICC2_lm_shell tool

It is also good to create NDM by source 00_create_ndm.tcl, but when you proceed for the first time, it is recommended that you open the script and apply it one phrase at a time

1. Invoke tool icc2_lm_shell

- **Recommend the icc2/2021.06-SP5 version**

>> icc2_lm_shell | tee -l ./00_create_ndm.log ;# (| tee -l ./00_create_ndm.log) is the command to output log

```
Library Manager (TM)

Version S-2021.06-SP5 for linux64 - Jan 14, 2022 -SLE

Copyright (c) 1988 - 2022 Synopsys, Inc.
This software and the associated documentation are proprietary to Synopsys,
Inc. This software may only be used in accordance with the terms and conditions
of a written license agreement with Synopsys, Inc. All other use, reproduction,
or distribution of this software is strictly prohibited. Licensed Products
communicate with Synopsys servers for the purpose of providing software
updates, detecting software piracy and verifying that customers are using
Licensed Products in conformity with the applicable License Key for such
Licensed Products. Synopsys will use information gathered in connection with
this process to deliver software updates and pursue software pirates and
infringers.

Inclusivity & Diversity - Visit SolvNetPlus to read the "Synopsys Statement on
Inclusivity and Diversity" (Refer to article 000036315 at
https://solvnetplus.synopsys.com)

lm_shell>
```

2. icc2_lm_shell> source \$WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/01_source_design_setup.tcl

- **set step & source design_setup.tcl**

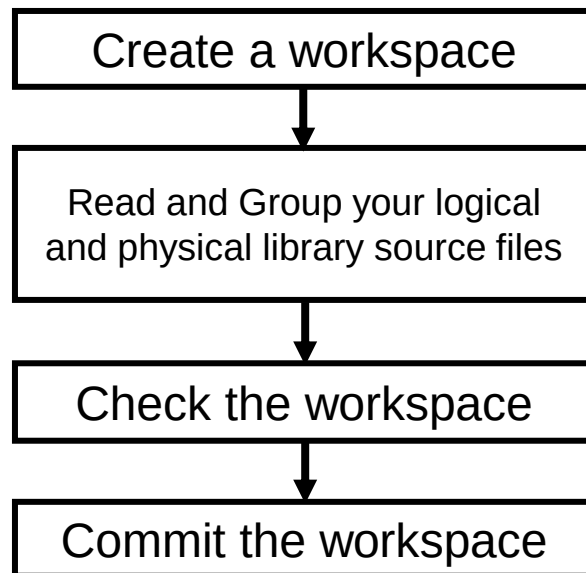
```
icc2_lm_shell> set step "ndm_preperation"
ndm_preperation
icc2_lm_shell>
icc2_lm_shell> source -echo -v ./icc2 common env/design_setup.tcl
```

Create NDM : Introduction to ICC2_Im_shell

- **What is IC compiler II Library Manager**

The library manager is used to import and validate logical and physical libraries for the place and route application. There are just a few key commands to understand when running the flow, which are outlined below

- **Running Flow**



Create NDM : Use ICC2_lm_shell tool

3. `icc2_lm_shell> source $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/02_source_set_app_option.tcl`

- `set sh_continue_on_error true & sourcing ndm_preperation.tcl`

```
icc2_lm_shell> set sh_continue_on_error true
true
icc2_lm_shell> #set old [sh date t%m%d_%H:%M:%S]
icc2_lm_shell>
icc2_lm_shell> source -echo -v ./icc2_common_env/ndm_preperation.tcl
```

4. `icc2_lm_shell> source $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/03_create_workspace.tcl`

- `set .tf, .lef, .db file path`

`icc2_lm_shell> set lib_tech "$data(tech)"`

`icc2_lm_shell> set lib_lef "$data(lefs)"`

`icc2_lm_shell> set lib_db "$data(dbs)"`

```
icc2_lm_shell> set lib_tech "$data(tech)"
/slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/TECH/saed32nm_1p9m_mw.tf
icc2_lm_shell> set lib_lef "$data(lefs)"
/slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32nm_rvt_1p9m.lef /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32nm_l
vt_1p9m.lef /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32nm_hvt_1p9m.lef /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LE
F/saed32_io_fc_all.lef /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32_PLL.lef /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdi
r/LEF/saed32sram.lef
icc2_lm_shell> set lib_db "$data(dbs)"
/slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ff1p16v125c_2p75v.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS
/saed32io_fc_ff1p16vn40c_2p75v.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ss0p95v125c_2p25v.db /slowfs/dgscratc
h2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ss0p95vn40c_2p25v.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32pll_ff1p1
6v125c_2p75v.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32pll_ff1p16vn40c_2p75v.db /slowfs/dgscratch2/asic/2_BACKEND/1
_pnr/libdir/DBS/saed32pll_ss0p95v125c_2p25v.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32pll_ss0p95vn40c_2p25v.db /slowf
s/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32sram_ff1p16v125c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32sram_f
f1p16vn40c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32sram_ss0p95v125c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/lib
dir/DBS/saed32sram_ss0p95vn40c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32hvt_ff1p16v125c.db /slowfs/dgscratch2/asic/
2_BACKEND/1_pnr/libdir/DBS/saed32hvt_ff1p16vn40c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32hvt_ss0p95v125c.db /slowf
s/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32hvt_ss0p95vn40c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32lvt_ff1
p16v125c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32lvt_ff1p16vn40c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir
/DBS/saed32lvt_ss0p95v125c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32lvt_ss0p95vn40c.db /slowfs/dgscratch2/asic/2_BA
CKEND/1_pnr/libdir/DBS/saed32rvt_ss0p95vn40c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32rvt_ff1p16v125c.db /slowfs/dg
scratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32rvt_ff1p16vn40c.db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32rvt_ss0p95v
125c.db
```

Create NDM : Use ICC2_lm_shell tool

- **create_workspace**

```
icc2_lm_shell> if {[length $lib_tech] == 0} {  
?   echo "Error: technology file not found, aborted"  
?   return  
? }  
icc2_lm_shell>  
icc2_lm_shell> create_workspace -flow exploration \  
?   -technology $lib_tech EXPLORE -scale_factor 1000  
Warning: Layer 'VIA1' 'onGrid' and 'onWireTrack' coexist. (TECH-025)  
Information: Loading db file '/slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/TECH/saed32nm_1p9m_mw.tf'  
... created workspace EXPLORE in exploration flow
```

```
Information: Loading db file '/slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32rvt_ss0p95v125c.db' (FILE-007)  
{saed32io_fc_ff1p16v125c_2p75v saed32io_fc_ff1p16vn40c_2p75v saed32io_fc_ss0p95v125c_2p25v saed32io_fc_ss0p95vn40c_2p25v saed32pll_ff1  
p16v125c_2p75v saed32pll_ff1p16vn40c_2p75v saed32pll_ss0p95v125c_2p25v saed32pll_ss0p95vn40c_2p25v saed32sram_ff1p16v125c saed32sram_f  
f1p16vn40c saed32sram_ss0p95v125c saed32sram_ss0p95vn40c saed32hvt_ff1p16v125c saed32hvt_ff1p16vn40c saed32hvt_ss0p95v125c saed32hvt_s  
s0p95vn40c saed32lvt_ff1p16v125c saed32lvt_ff1p16vn40c saed32lvt_ss0p95v125c saed32lvt_ss0p95vn40c saed32rvt_ss0p95vn40c saed32rvt_ff1  
p16v125c saed32rvt_ff1p16vn40c saed32rvt_ss0p95v125c}
```

❖ If .db files are read correctly, the list of dbs read as above is output

→ This syntax is to output and return the following error message if there is no .tf file

5. `icc2_lm_shell> source $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/04_read_db.tcl`

- **read library db files**

```
icc2_lm_shell> echo "create workspace was done at [date]"  
create_workspace was done at Wed Jun 28 19:34:07 2023  
icc2_lm_shell>  
icc2_lm_shell> if {$lib_db != " "} {  
?   read_db $lib_db  
? }
```

Create NDM : Use ICC2_lm_shell tool

6. `icc2_lm_shell> source $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/05_read_lef.tcl`

- read .lef files

```
icc2_lm_shell> set fout [open $out_file "w"]
file18
icc2_lm_shell> puts $fout "LAYER OVERLAP"
icc2_lm_shell> puts $fout "END OVERLAP"
icc2_lm_shell> close $fout
icc2_lm_shell> set overlap_lef $out_file
./OVERLAP.lef
icc2_lm_shell> if {$lib_lef != ""} {
?   read_lef $overlap_lef
?   read_lef $lib_lef
? }
```

if \$lib_lef is not empty read \$overlap_lef & \$lib_lef

save "LAYER OVERLAP & END OVERLAP" to OVERLAP.lef file

❖ If .lef files are read correctly, the lists of lef are display as below

```
{saed32nm_rvt_1p9m saed32nm_lvt_1p9m saed32nm_hvt_1p9m saed32_io_fc_all saed32_PLL saed32sram}
```

Create NDM : Use ICC2_lm_shell tool

7. `icc2_lm_shell> source $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/06_group_libs.tcl`

- **group_libs**

```
icc2_lm_shell> group_libs
Information: Identified a potential valid scaling group. (LM-027)
Information: Identified a potential valid scaling group. (LM-027)
Information: Identified a potential valid scaling group. (LM-027)
Information: Identified a potential valid scaling group. (LM-027)
Information: Identified a potential valid scaling group. (LM-027)
Information: Identified a potential valid scaling group. (LM-027)
...Created 7 lib groups
...Created sub-workspace saed32io_fc with type normal: 4 logical libs, 1 physical lib, 0 scaling groups.
...Created sub-workspace saed32pll with type normal: 4 logical libs, 1 physical lib, 0 scaling groups.
...Created sub-workspace saed32sram with type normal: 4 logical libs, 1 physical lib, 0 scaling groups.
...Created sub-workspace saed32hvt with type normal: 4 logical libs, 1 physical lib, 0 scaling groups.
...Created sub-workspace saed32lvt with type normal: 4 logical libs, 1 physical lib, 0 scaling groups.
...Created sub-workspace saed32rvt with type normal: 4 logical libs, 1 physical lib, 0 scaling groups.
...Created sub-workspace EXPLORE_physical_only with type physical_only.
...Created 7 sub-workspaces
```

→ Create groups of {saed32io saed32pll saed32sram saed32hvt saed32lvt saed32rvt }

- **Write_exploration.tcl**

Write exploration.tcl in your run directory using write_workspace command

```
icc2_lm_shell> write_workspace -file ${NDM_DIR}/exploration.tcl
```

Create NDM : Use ICC2_lm_shell tool

8. `icc2_lm_shell> source $WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/07_report_and_check_workspace.tcl`

- **Report_workspace**

```
icc2_lm_shell> report_workspace

Workspace name: EXPLORE

...Flow:          exploration

...Checked?       No

...Technology file:
    /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/TECH/saed32nm_1p9m_mw.tf

...Current PVT Configuration:
    <No rules specified>

...DB/Liberty files:
  1. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ff1p16v125c_2p75v.db
  2. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ff1p16vn40c_2p75v.db
  3. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ss0p95v125c_2p25v.db
  4. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ss0p95vn40c_2p25v.db
  5. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32pll_ff1p16v125c_2p75v.db
  6. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32pll_ff1p16vn40c_2p75v.db
  7. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32pll_ss0p95v125c_2p25v.db
  8. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32pll_ss0p95vn40c_2p25v.db
  9. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32sram_ff1p16v125c.db
 10. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32sram_ff1p16vn40c.db
 11. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32sram_ss0p95v125c.db
 12. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32sram_ss0p95vn40c.db
 13. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32hvt_ff1p16v125c.db
 14. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32hvt_ff1p16vn40c.db
 15. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32hvt_ss0p95v125c.db
 16. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32hvt_ss0p95vn40c.db
 17. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32lvt_ff1p16v125c.db
 18. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32lvt_ff1p16vn40c.db
 19. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32lvt_ss0p95v125c.db
 20. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32lvt_ss0p95vn40c.db
 21. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32rvt_ss0p95v125c.db
 22. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32rvt_ff1p16v125c.db
 23. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32rvt_ff1p16vn40c.db
 24. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32rvt_ss0p95v125c.db

...Physical source files:
  1. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32nm_rvt_1p9m.lef
  2. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32nm_lvt_1p9m.lef
  3. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32nm_hvt_1p9m.lef
  4. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32_io_fc_all.lef
  5. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32_PLL.lef
  6. /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32sram.lef

...TLUP files: <none>

...Library groups: 7

  1. Group name: "saed32io_fc"
     Logical libs: 4
     Physical libs: 1
     Lib-cells: 65

  2. Group name: "saed32pll"
     Logical libs: 4
     Physical libs: 1
     Lib-cells: 1

  3. Group name: "saed32sram"
     Logical libs: 4
     Physical libs: 1
     Lib-cells: 35

  4. Group name: "saed32hvt"
     Logical libs: 4
     Physical libs: 1
     Lib-cells: 294

  5. Group name: "saed32lvt"
     Logical libs: 4
     Physical libs: 1
     Lib-cells: 294

  6. Group name: "saed32rvt"
     Logical libs: 4
     Physical libs: 1
     Lib-cells: 294

  7. Group name: "EXPLORE_physical_only" (physical only)
     Physical libs: 3
     Lib-cells: 168
```

Create NDM : Use ICC2_lm_shell tool

- Check workspace : check the workspace before commit workspace

```
icc2_lm_shell> check workspace
```

```
Workspace check succeeded!  
Information: 7/7 workspaces checked successfully (LMUI-044)
```

9. icc2_lm_shell> source \$WORK_DIR/2_BACKEND/1_pnr/rundir/00_create_ndm/script/08_commit_and_remove_workspace.tcl

- Commit workspace : save ndm library to your run directory

```
icc2_lm_shell> commit workspace
```

```
Warning: The command commit_workspace (in the EXPLORATION flow) is deprecated. Use process_workspaces.  
... wrote saed32io_fc.ndm  
... removing library 'saed32io_fc_fflp16v125c_2p75v'  
... removing library 'saed32io_fc_fflp16vn40c_2p75v'  
... removing library 'saed32io_fc_ss0p95v125c_2p25v'  
... removing library 'saed32io_fc_ss0p95vn40c_2p25v'  
... removing library 'saed32_io_fc_all'  
... lib workspace removed!
```

Create NDM : Use ICC2_lm_shell tool

- **Remove_workspace** : remove workspace in memory

```
icc2_lm_shell> remove_workspace
... removing lib groups
... removing library 'saed32io_fc_ff1p16v125c_2p75v'
... removing library 'saed32io_fc_ff1p16vn40c_2p75v'
... removing library 'saed32io_fc_ss0p95v125c_2p25v'
... removing library 'saed32io_fc_ss0p95vn40c_2p25v'
... removing library 'saed32pll_ff1p16v125c_2p75v'
... removing library 'saed32pll_ff1p16vn40c_2p75v'
... removing library 'saed32pll_ss0p95v125c_2p25v'
... removing library 'saed32pll_ss0p95vn40c_2p25v'
... removing library 'saed32sram_ff1p16v125c'
... removing library 'saed32sram_ff1p16vn40c'
... removing library 'saed32sram_ss0p95v125c'
... removing library 'saed32sram_ss0p95vn40c'
... removing library 'saed32hvt_ff1p16v125c'
... removing library 'saed32hvt_ff1p16vn40c'
... removing library 'saed32hvt_ss0p95v125c'
... removing library 'saed32hvt_ss0p95vn40c'
... removing library 'saed32lvt_ff1p16v125c'
... removing library 'saed32lvt_ff1p16vn40c'
... removing library 'saed32lvt_ss0p95v125c'
... removing library 'saed32lvt_ss0p95vn40c'
... removing library 'saed32rvt_ss0p95vn40c'
... removing library 'saed32rvt_ff1p16v125c'
... removing library 'saed32rvt_ff1p16vn40c'
... removing library 'saed32rvt_ss0p95v125c'
... removing library 'saed32nm_rvt_1p9m'
... removing library 'saed32nm_lvt_1p9m'
... removing library 'saed32nm_hvt_1p9m'
... removing library 'saed32_io_fc_all'
... removing library 'saed32_PLL'
... removing library 'saed32sram'
... lib workspace removed!
```

Create NDM : Use ICC2_lm_shell tool

- Move ndm files to your ndm directory

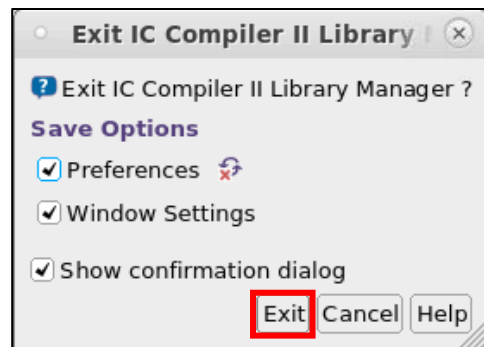
```
icc2_lm_shell> sh mv *.ndm ${NDM_DIR}
```

check_workspace.ems	icc2_command.log	icc2_lm_output.txt	ndm_OLD	PreFrameCheck	saed32lvt.ndm	saed32sram.ndm
EXPLORE_physical_only.ndm	icc2_common_env	icc2_output.txt	nlib	saed32hvt.ndm	saed32pll.ndm	script
FP_data	icc2_lm_command.log	ndm	OVERLAP.lef	saed32io_fc.ndm	saed32rvt.ndm	user_script

```
< user@linux ndm >> pwd
/slowfs/dgscratch2/asic/ASIC_PROJECT/2_BACKEND/1_pnr/rundir/00_create_ndm/ndm
< user@linux ndm >> ls
exploration.tcl      saed32hvt.ndm      saed32lvt.ndm      saed32rvt.ndm
EXPLORE_physical_only.ndm saed32io_fc.ndm saed32pll.ndm      saed32sram.ndm
```

- Exit icc2_lm_shell

```
icc2_lm_shell> exit
```



When you finished, use exit command to exit the tool

Create NDM : Check if the NDM files are created correctly

1. setenv WORK_DIR "your working directory"
2. cd \${WORK_DIR}/asic/2_BACKEND/1_pnr/rundir/00_create_ndm/ndm - move ndm directory
3. gvim exploration.tcl - the exploration.tcl is display the below for example

```
set_app_options -as_user_default -name lib.physical_model.block_all -value true
set_app_options -name file.lef.allow_site_conflicts -value true
set_app_options -name file.lef.auto_rename_conflict_sites -value true
set_app_options -name lib.workspace.allow_commit_workspace_overwrite -value true
set_app_options -name lib.workspace.group_libs_create_slg -value false
set_app_options -name lib.workspace.allow_missing_related_pg_pins -value true
set_app_options -name lib.logic_model.auto_remove_incompatible_timing_designs -value true
set_app_options -name lib.workspace.remove_frame_bus_properties -value true
set_app_options -name lib.logic_model.require_same_opt_attrs -value false
set_app_options -name lib.logic_model.use_db_rail_names -value true
set_app_options -name lib.logic_model.auto_remove_timing_only_designs -value true
set_app_options -name lib.workspace.save_design_views -value false
set_app_options -name lib.workspace.save_layout_views -value false
set_app_options -name file.lef.non_real_cut_obs_mode -value true
set_app_options -name lib.workspace.group_libs_macro_grouping_strategy -value single_cell_per_lib
set_app_options -name lib.workspace.group_libs_naming_strategies -value common_prefix
set_app_options -name lib.workspace.group_libs_fix_cell_shadowing -value false
# workspace saed32io_fc:
create_workspace -scale_factor 1000 -technology /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/TECH/saed32nm_1p9m_mw.tf saed32io_fc
read_db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_fflp16v125c_2p75v.db
read_db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_fflp16vn40c_2p75v.db
read_db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ss0p95v125c_2p25v.db
read_db /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/DBS/saed32io_fc_ss0p95vn40c_2p25v.db
read_lef /slowfs/dgscratch2/asic/2_BACKEND/1_pnr/libdir/LEF/saed32_io_fc_all.lef
check_workspace
commit_workspace -output ./saed32io_fc.ndm
remove_workspace
```

App option lists used to create NDM files

List of tf, lef, db files used in the workspace

Thank You

